

V5 · EDIÇÃO FINAL · WORKBOOK HANDS-ON · CURSOR-FIRST

Prepare a expectativa. Execute o cenário. Preserve a evidência.

Laboratórios, templates e checkpoints reproduzíveis.

```
repository/.cursor/  
├─ rules/ · agents/ · skills/  
├─ playbooks/ · knowledge/  
├─ contracts/ · evals/  
└─ evidence/ · hooks/
```

Sumário

0.1	Pré-requisitos	1
0.2	Ordem recomendada	1
1	Laboratório 1 — contexto mínimo executável	2
1.1	Montagem do experimento	2
1.1.1	1. Crie a Project Rule	2
1.1.2	2. Crie o subagent	3
1.1.3	3. Crie a Skill	3
1.1.4	4. Crie o Contract	4
1.1.5	O que este experimento pretende provar	4
1.1.6	5. Registre a expectativa antes da execução	5
1.1.7	6. Execute o teste no Cursor	6
1.1.8	7. Compare a resposta com o resultado esperado	8
1.1.9	8. Registre a evidência	9
1.1.10	9. Interprete o resultado	10
1.1.11	10. Depois do wiring, teste a descoberta	12
1.2	Do arquivo backend caótico à separação	14
1.3	Agent: o conteúdo HOT	15
1.3.1	Teste mínimo do Agent HOT	17
1.4	Playbook: coordenação WARM	17
1.4.1	Teste mínimo do Playbook WARM	19
1.5	Skill: capacidade especializada sob demanda	19
1.5.1	Teste mínimo da Skill	21
1.6	Knowledge: fundamentação WARM ou COLD	21
1.6.1	Teste mínimo do Knowledge	23
1.7	Anatomia dos pacotes	23
1.8	YAML frontmatter: descoberta, roteamento e governança	24
1.8.1	id e name resolvem problemas diferentes	25
1.9	Frontmatter de uma Skill	26
1.9.1	Frontmatter de um Playbook	27
1.9.2	Frontmatter de uma Rule	27
2	Rules: obrigação, proibição e fallback	30
2.1	Engineering Rule: persistir antes de notificar	30
2.1.1	E uma Domain Rule?	31
2.1.2	Evitando drift entre a Rule e os artefatos relacionados	32
2.1.3	Teste mínimo da Rule	33
2.2	Contracts: o encaixe verificável	34
2.3	Assertions: nomear o que será verificado	37
2.4	Evals: verificar recuperação e aplicação	39
2.4.1	Camadas de avaliação	39
2.4.2	De onde vêm os valores esperados	40
2.4.3	Eval Spec	41
2.4.4	Classe da avaliação e mecanismo de prova	43
2.4.5	Como aplicar na prática	43

2.4.6	Objetivo SMART do checkpoint	44
2.5	Evidence Records: tornar a execução auditável	44
2.5.1	Quando usar	45
2.5.2	Exercício copiável: auditar uma execução	46
2.5.3	Exemplo preenchido do exercício	48
2.5.4	Exemplo de saída	50
2.5.5	Como interpretar os resultados	52
2.5.6	Diagnóstico por camada	53
2.5.7	Conteúdo mínimo e retenção	54
2.5.8	Checklist de qualidade do Evidence Record	54
2.6	Reduzindo omissão e alucinação operacional	54
2.7	Fechando o capítulo: do pedido à evidência	55
3	Laboratório 2 — estudo de caso: operação assíncrona	57
3.1	O requisito	58
3.2	Contrato de aceitação	58
3.2.1	Condições verificáveis da aceitação	59
3.3	Contrato de progresso e conclusão	59
3.3.1	Condições verificáveis da conclusão	60
3.4	Estados e transições	60
3.5	Edge cases obrigatórios neste recorte	61
3.6	Massa progressiva de testes	61
3.7	Captura operacional observada	62
3.8	Fechando o ciclo do caso	62
3.8.1	Eval Spec do caso	63
3.8.2	Evidence Record do caso	64
4	Laboratório 3 — kits e templates práticos	67
4.1	Kit A - contexto sob avaliação	67
4.1.1	Ordem de execução	68
4.1.2	Definition of Done do Kit A	68
4.2	Kit B - harness de avaliação	68
4.2.1	Agent de avaliação preenchido	69
4.2.2	Skill preenchida	71
4.2.3	Variações de Assertions	73
4.2.4	Execução do kit	74
4.3	Template mínimo de Agent	74
4.4	Template mínimo de Skill	75
4.5	Anatomia ampliada opcional de uma Skill	76
4.5.1	Corpo recomendado	77
4.5.2	Exemplo compacto do corpo	77
4.6	Template de Playbook	79
4.7	Template de Rule	80
4.8	Template de Contract	81
4.9	Template de Eval	81
4.10	Template de Evidence Record	82
5	Apêndice — Laboratórios e checkpoints práticos	84
5.1	Laboratório — Agent HOT	84
5.1.1	1. Registre a expectativa antes de executar	85
5.1.2	2. Execute o Run 001 — roteamento	85
5.1.3	3. Preserve o resultado do Run 001	86
5.1.4	4. Avalie o Run 001	86
5.1.5	5. Execute o Run 002 — condição de parada	87
5.1.6	6. Preserve o resultado do Run 002	88
5.1.7	7. Avalie o Run 002	88
5.1.8	8. Registre as avaliações	88
5.1.9	9. Interprete o checkpoint	89

5.2	Checklist do curador	90
5.2.1	Estrutura	90
5.2.2	Recuperação	90
5.2.3	Aplicação	90
5.2.4	Operação	90
6	Referências técnicas	91
6.1	Especificações e guias	91
6.2	Fundamentos de aprendizagem e contexto	91
6.3	Tecnologias do estudo de caso	91

Como usar este workbook

Este material é o companion hands-on do livro **Context Engineering para Times de Desenvolvimento**. Ele preserva os prompts, schemas, templates, massas e execuções que interrompiam a rota conceitual. Para cada laboratório, registre a expectativa antes de executar, diferencie resultado funcional de lineage e use **INCONCLUSIVE** quando a evidência não permitir uma conclusão.

0.1 Pré-requisitos

- ter lido ao menos a rota essencial do livro principal;
- usar uma cópia de trabalho do diretório `lab/`;
- registrar host, versão, modelo e data da execução;
- não utilizar respostas anteriores como massa visível ao agente sob avaliação;
- interromper quando faltar fonte aprovada, observabilidade ou autoridade para a próxima ação.

0.2 Ordem recomendada

1. Experimento mínimo executável.
2. Artefatos e testes focados.
3. Contract, Assertion, Eval e Evidence Record.
4. Estudo assíncrono integrado.
5. Kits, templates e laboratório Agent HOT.
6. Checklist do curador.

Cada PASS vale apenas para o cenário, a versão e os verificadores registrados. O workbook ensina um processo de produção de evidência; não promete comportamento determinístico de uma LLM.

Capítulo 1

Laboratório 1 — contexto mínimo executável

1.1 Montagem do experimento

Antes de conhecer o catálogo inteiro, monte um experimento pequeno o suficiente para compreender **o que cada artefato contém, como eles se relacionam, como participam da execução e como verificar o resultado**.

O laboratório utiliza quatro arquivos de entrada. A estrutura aplica a regra apresentada anteriormente: como o Contract deste exemplo pertence exclusivamente à capacidade `validate-api-contract`, ele permanece junto da própria Skill.

```
.cursor/
├─ rules/
│   └─ backend-safety.mdc
├─ agents/
│   └─ backend-development.md
└─ skills/
    └─ validate-api-contract/
        ├── SKILL.md
        └─ contracts/
            └─ report-request.contract.yaml
```

A árvore mostra duas relações importantes. A Rule e o subagent participam por mecanismos próprios do host, enquanto o Contract é um recurso local da Skill e precisa ser alcançado a partir dela. Sua posição dentro de `validate-api-contract/` torna explícito esse ownership e evita promover para o escopo compartilhado um artefato utilizado apenas por essa capacidade.

A estrutura, porém, mostra apenas **onde os arquivos ficam e como estão organizados**. Para tornar o experimento reproduzível, o próximo passo é preencher cada arquivo com o conteúdo mínimo necessário e tornar explícitas as conexões entre eles.

1.1.1 1. Crie a Project Rule

Arquivo `.cursor/rules/backend-safety.mdc`:

```
---
description: Regras mínimas de segurança para operações backend assíncronas.
globs:
alwaysApply: true
---

# Backend Safety
```

- ``202 Accepted`` informa que a solicitação foi aceita, não que o processamento terminou.
- O estado da operação deve permanecer em uma fonte durável e consultável.
- Uma notificação em tempo real não substitui a fonte canônica do estado.
- O cliente deve possuir um mecanismo para recuperar o estado após uma desconexão.

Neste experimento, a Rule fornece invariantes que devem permanecer verdadeiras independentemente da solução proposta.

1.1.2 2. Crie o subagent

Arquivo `.cursor/agents/backend-development.md`:

```
---
name: backend-development
description: Analisa tarefas backend e coordena propostas de contratos para operações assíncronas.
---

# Backend Development

## Missão

Propor soluções backend que preservem estado, recuperabilidade e contratos verificáveis.

## Roteamento

Quando a tarefa envolver criação ou revisão de contrato HTTP:

1. utilize a skill `validate-api-contract`;
2. considere as regras aplicáveis do repositório;
3. não conclua a tarefa enquanto houver violação crítica identificada.

## Saída

Informe:

- contrato HTTP proposto;
- ordem das operações;
- estratégia de recuperação do estado;
- artefatos utilizados para sustentar a proposta.
```

O subagent coordena a tarefa. Ele não substitui a Skill nem o Contract: sua responsabilidade é encaminhar a execução para a capacidade adequada.

1.1.3 3. Crie a Skill

Arquivo `.cursor/skills/validate-api-contract/SKILL.md`:

```
---
name: validate-api-contract
description: Valida contratos HTTP para operações assíncronas que precisam preservar estado e permitir recuperação pelo cliente.
---

# Validate API Contract

Quando a tarefa envolver uma operação HTTP que pode continuar depois da resposta inicial:
```

1. leia ``contracts/report-request.contract.yaml``;
2. compare a proposta com as entradas, saídas e pós-condições declaradas;
3. verifique se existe estado durável consultável;
4. verifique se o cliente consegue reconciliar o estado após uma desconexão;
5. rejeite propostas que tratem notificação em tempo real como fonte canônica do estado;
6. informe o Contract utilizado na validação.

A Skill cria a ligação explícita com um recurso que pertence à própria capacidade. Como o Contract está dentro do diretório `validate-api-contract/`, o `SKILL.md` pode referenciá-lo por um caminho relativo, mantendo juntos a capacidade e o recurso utilizado por ela.

A existência do arquivo dentro da Skill, porém, não substitui essa referência: o diretório `contracts/` continua sendo uma convenção de organização deste livro e não representa, por si só, um mecanismo automático de carregamento.

1.1.4 4. Crie o Contract

Arquivo `.cursor/skills/validate-api-contract/contracts/report-request.contract.yaml`:

```
id: report-request
version: 1.0.0

operation:
  method: POST
  path: /reports

response:
  status: 202
  body:
    jobId: string
    statusUrl: string

postconditions:
  - O job deve possuir estado durável e consultável.
  - A resposta 202 não representa conclusão do processamento.
  - A conclusão deve ser persistida antes de uma notificação em tempo real.

recovery:
  source_of_truth: statusUrl
  realtime_notification: optional
```

O Contract declara os critérios que a Skill utilizará para avaliar a proposta. Neste experimento, ele pertence exclusivamente à capacidade `validate-api-contract`; por isso, permanece dentro do pacote da Skill em vez de ser promovido para `.cursor/contracts/`.

1.1.5 O que este experimento pretende provar

Os arquivos agora possuem responsabilidades e relações explícitas:

```
Pedido
↓
Project Rule fornece invariantes
↓
subagent coordena
↓
Skill valida
↓
Contract local fornece critérios
↓
```

Resposta

Esse fluxo representa a **expectativa do experimento**. A execução ainda precisa mostrar quais dessas transições realmente ocorreram.

i SMART em 30 segundos

SMART ajuda a transformar uma intenção em um objetivo verificável: **S**pecific (específico), **M**easurable (mensurável), **A**chievable (atingível), **R**elevant (relevante) e **T**ime-bound (limitado no tempo).

Neste checkpoint, queremos executar um cenário definido, observar critérios previamente estabelecidos e concluir a avaliação até o final da seção.

O objetivo SMART do checkpoint é: **até o final desta seção, executar o cenário no Cursor, avaliar o resultado funcional e registrar quais transições possuem evidência suficiente para serem confirmadas.**

1.1.6 5. Registre a expectativa antes da execução

Antes de executar o prompt, transforme os critérios do experimento em uma referência que possa ser comparada posteriormente com a resposta observada.

Esse arquivo não participa da execução do Agent. Ele pertence à avaliação do checkpoint e registra **o que deve ser verdadeiro antes de conhecermos a resposta do modelo**.

Crie:

```
.cursor/
├─ evals/
│   └─ checkpoints/
│       └─ contexto-minimo/
│           └─ expected.md
```

Arquivo `.cursor/evals/checkpoints/contexto-minimo/expected.md`:

```
# Contexto mínimo executável – expectativa
```

```
## Objetivo
```

Avaliar se uma proposta para iniciar uma geração de relatório assíncrona preserva estado durável, permite recuperação pelo cliente e utiliza corretamente os artefatos definidos no experimento.

```
## Resultado funcional esperado
```

- ``202 Accepted`` representa aceitação da solicitação, não conclusão do processamento;
- a resposta contém um identificador da operação, como ``jobId``;
- a resposta fornece um recurso consultável, como ``statusUrl``;
- o estado da operação permanece em uma fonte durável e recuperável;
- o estado final é persistido antes da emissão de uma notificação de conclusão;
- após uma desconexão, o cliente consegue reconciliar o estado pela fonte consultável;
- SignalR ou mecanismo equivalente funciona como canal de notificação, não como fonte canônica do estado;
- as entradas e pós-condições do Contract são preservadas.

```
## Transições esperadas
```

- Pedido → Project Rule;
- Pedido → subagent ``backend-development``;
- subagent → Skill ``validate-api-contract``;

- Skill → ``contracts/report-request.contract.yaml``;
- artefatos aplicáveis → resposta final.

Condições de reprovação funcional

Considere o resultado funcional inadequado se a proposta:

- tratar ``202 Accepted`` como conclusão da geração;
- depender exclusivamente de uma notificação em tempo real para descobrir o estado final;
- não oferecer uma forma durável de consultar o estado;
- emitir a notificação de conclusão antes de persistir o estado correspondente;
- contradizer uma pós-condição do Contract.

Observabilidade

A correção da resposta não prova que todas as transições esperadas ocorreram. A participação de cada artefato deverá ser avaliada separadamente com base nas evidências disponíveis após a execução.

Esse arquivo funciona como a **expectativa do experimento**. Ele deve permanecer inalterado durante a rodada que será avaliada; se os critérios precisarem mudar, atualize a expectativa antes de iniciar uma nova execução.

A partir daqui, o laboratório possui três momentos claramente separados:

```

expected.md
expectativa definida antes da execução
      ↓
    execução no Cursor
      ↓
resposta e evidências observadas
      ↓
    comparação e verdict
  
```

A existência de `expected.md` também evita que uma resposta convincente do Agent altere retrospectivamente aquilo que o experimento consideraria correto.

1.1.7 6. Execute o teste no Cursor

Com os artefatos de execução e a expectativa registrados, o próximo passo é verificar primeiro o **wiring básico**.

Neste livro, **wiring** é a conexão entre os componentes necessários para que uma execução consiga percorrer o caminho esperado. Neste experimento, significa verificar se o Agent principal consegue delegar a tarefa ao subagent, se o subagent consegue encaminhá-la para a Skill e se a Skill consegue alcançar o Contract que utiliza.

```

Agent principal
  ↓
subagent backend-development
  ↓
Skill validate-api-contract
  ↓
Contract local
  
```

Neste primeiro teste, não queremos descobrir se o sistema consegue encontrar esse caminho sozinho. Queremos responder a uma pergunta mais simples:

O caminho funciona quando indicamos explicitamente por onde a execução deve começar?

Abra a raiz do repositório no Cursor e inicie **uma nova conversa no Agent**, evitando reutilizar uma conversa que já contenha contexto de outros testes. O isolamento reduz a possibilidade de uma resposta anterior influenciar o resultado observado.

Neste teste controlado, a escolha do subagent será explícita. Isso é intencional: primeiro verificamos o **wiring**; somente depois avaliaremos **discovery**, isto é, se o sistema consegue reconhecer a tarefa e encontrar os recursos adequados sem que o caminho seja informado diretamente.

Cole o prompt a seguir **na conversa principal do Agent no Cursor**:

Tarefa

Objetivo

Propor o contrato para iniciar uma geração de relatório que pode durar minutos.

Execução

Use o subagent ``backend-development`` para coordenar a proposta. A partir das instruções desse subagent, utilize a Skill e os recursos aplicáveis à validação do contrato.

Restrições

- Não trate a resposta inicial como conclusão do processamento.
- Não trate notificação em tempo real como fonte durável do estado.
- Informe os artefatos que sustentaram a proposta.

Saída esperada

1. Contrato HTTP mínimo.
2. Ordem das operações.
3. Fallback para reconexão do front-end.
4. Artefatos utilizados.

A execução parte do **Agent principal**, que recebe o prompt e deve delegar a atividade ao subagent backend-development. No Cursor, custom subagents definidos em `.cursor/agents/` podem ser utilizados pelo agente principal para executar partes especializadas de uma tarefa.[^cursor-runtime]

A partir daí, o encadeamento esperado é:

```
Prompt no Agent principal
  ↓
delegação para backend-development
  ↓
instruções do subagent
  ↓
Skill validate-api-contract
  ↓
contracts/report-request.contract.yaml
  ↓
resposta final no Agent principal
```

Esse fluxo representa a **expectativa de wiring** registrada antes da execução. Ele não deve ser tratado como prova de que todas as transições ocorreram.

Ao final da execução, considere a resposta apresentada na conversa principal como o **resultado bruto do teste**. Registros adicionais de delegação, recuperação de arquivos ou uso de recursos só devem ser considerados evidência quando forem efetivamente observáveis no host.

Neste primeiro momento, responda apenas a duas perguntas:

1. o Agent conseguiu produzir uma resposta para o cenário usando o caminho controlado definido no prompt?
2. a resposta respeitou as restrições mínimas de durabilidade e recuperação?

A próxima etapa comparará esse resultado com a expectativa registrada em `expected.md`.

1.1.8 7. Compare a resposta com o resultado esperado

Com a execução concluída, abra novamente:

```
.cursor/evals/checkpoints/contexto-minimo/expected.md
```

Não altere seus critérios com base na resposta recebida. Use o arquivo como referência e compare **expectativa por expectativa** com o resultado produzido pelo Agent.

A comparação deve seguir esta direção:

```
expected.md
  ↓
critério previamente definido
  ↓
trecho correspondente da resposta
  ↓
diferença observada
  ↓
avaliação posterior
```

Comece pela seção **Resultado funcional esperado** de `expected.md`. Para cada item, procure na resposta do Cursor uma evidência correspondente.

Critério em <code>expected.md</code>	O que procurar na resposta
202 Accepted representa aceitação	A resposta não trata 202 como conclusão do processamento.
A operação possui identificação	Existe um identificador como <code>jobId</code> .
O estado pode ser recuperado	Existe um recurso consultável, como <code>statusUrl</code> .
O estado é durável	A proposta mantém o estado fora do canal de notificação.
A conclusão é persistida antes da notificação	A ordem das operações preserva essa sequência.
O cliente consegue se recuperar de uma desconexão	Existe consulta ou reconciliação posterior do estado.
SignalR não é fonte canônica	SignalR ou mecanismo equivalente aparece apenas como canal de notificação.
O Contract é respeitado	Entradas e pós-condições relevantes aparecem preservadas na solução.

Depois, utilize a seção **Transições esperadas** do mesmo arquivo para avaliar separadamente o lineage da execução.

Resultado funcional: verifica se a solução produzida corresponde às expectativas registradas antes da execução.

Lineage da execução: representa a cadeia de artefatos e transições que sustentaram o resultado, verificando se existe evidência suficiente para afirmar quais recursos realmente participaram da execução.

Essa separação evita duas conclusões incorretas:

```
resposta correta
≠
todos os artefatos esperados foram comprovadamente utilizados

artefatos utilizados
≠
resultado funcionalmente correto
```

Por isso, compare as duas dimensões de forma independente. Uma resposta pode estar funcionalmente correta mesmo quando a participação de algum artefato não puder ser comprovada; da mesma forma, o uso observável de Rule, subagent, Skill e Contract não garante que a solução final respeitou todos os critérios esperados.

Nesta etapa, identifique **o que era esperado, o que foi observado e onde existem divergências**. A classificação formal em PASS, FAIL ou INCONCLUSIVE e o registro dessas evidências serão feitos na próxima etapa.

1.1.9 8. Registre a evidência

Até aqui, o laboratório produziu três tipos de informação que precisam permanecer separados:

```
artefatos de execução
→ Rule, subagent, Skill e Contract

expectativa definida antes da execução
→ evals/checkpoints/contexto-minimo/expected.md

resultado observado
→ resposta produzida pelo Cursor
```

Agora transforme o resultado observado e sua avaliação em evidência persistente, sem alterar a expectativa utilizada na rodada.

Crie:

```
.cursor/
├── evidence/
│   ├── checkpoints/
│   │   ├── contexto-minimo/
│   │   │   ├── run-001-response.md
│   │   │   └── run-001.md
```

Copie a resposta obtida na conversa do Cursor para `run-001-response.md` **sem corrigi-la ou reescrevê-la**. Esse arquivo preserva o resultado bruto observado na execução.

Em `run-001.md`, registre a comparação entre `expected.md` e o resultado observado:

```
# Contexto mínimo executável – Run 001

## Execução

- Host: Cursor
- Data: YYYY-MM-DD
- Expectativa: `.cursor/evals/checkpoints/contexto-minimo/expected.md`
- Resposta observada: `run-001-response.md`

## Resultado funcional

| Critério esperado | Evidência observada | Resultado |
|---|---|---
```

```

| `202 Accepted` representa aceitação | descrever trecho ou ausência | PASS/FAIL |
| Retorna identificador da operação | descrever trecho ou ausência | PASS/FAIL |
| Expõe estado consultável | descrever trecho ou ausência | PASS/FAIL |
| Preserva estado durável | descrever trecho ou ausência | PASS/FAIL |
| Persiste conclusão antes da notificação | descrever trecho ou ausência | PASS/FAIL |
| Permite recuperação após desconexão | descrever trecho ou ausência | PASS/FAIL |
| Não trata SignalR como fonte canônica | descrever trecho ou ausência | PASS/FAIL |
| Respeita o Contract | descrever trecho ou ausência | PASS/FAIL |

## Lineage

| Transição esperada | Evidência observada | Resultado |
|---|---|---|
| Pedido → Project Rule | descrever evidência | PASS/FAIL/INCONCLUSIVE |
| Pedido → subagent `backend-development` | descrever evidência | PASS/FAIL/INCONCLUSIVE |
| subagent → Skill `validate-api-contract` | descrever evidência | PASS/FAIL/INCONCLUSIVE |
| Skill → `contracts/report-request.contract.yaml` | descrever evidência |
PASS/FAIL/INCONCLUSIVE |

## Conclusão

- Resultado funcional: PASS/FAIL
- Lineage: PASS/FAIL/INCONCLUSIVE
- Observações:

```

Os resultados possuem significados diferentes.

PASS indica que a evidência observada sustenta o comportamento ou a transição esperada.

FAIL indica que o resultado observado contradiz um critério funcional ou que existe evidência suficiente para demonstrar que uma transição esperada não ocorreu como definida.

INCONCLUSIVE deve ser utilizado no lineage quando a evidência disponível não permite confirmar nem negar uma transição.

Por exemplo, a resposta pode respeitar todas as pós-condições do Contract sem que o host forneça evidência suficiente para demonstrar que `contracts/report-request.contract.yaml` foi realmente recuperado. Nesse caso, registre:

```

Resultado funcional: PASS
Lineage da Skill → Contract: INCONCLUSIVE

```

Não transforme a correção da resposta em prova de utilização do arquivo.

Ao final desta etapa, preserve os três elementos da rodada:

```

expected.md
→ o que deveria acontecer

run-001-response.md
→ o que o Agent produziu

run-001.md
→ como esperado e observado foram comparados

```

Essa separação permite revisar posteriormente a execução sem reconstruir a avaliação a partir da memória ou da resposta final.

1.1.10 9. Interprete o resultado

Com `run-001.md` preenchido, use o resultado funcional e o lineage para decidir **qual é a próxima ação**. Não trate todos os resultados negativos da mesma forma: uma falha funcional, uma falha

de wiring e uma limitação de observabilidade exigem correções diferentes.

Os principais cenários são:

Cenário	Resultado	Próxima ação
Solução correta e transições comprovadas	PASS funcional e PASS de lineage	Preserve a rodada e avance para o teste de descoberta.
Solução correta, mas a participação de algum artefato não pode ser comprovada	PASS funcional e INCONCLUSIVE de lineage	Preserve a limitação; não afirme lineage e melhore a observabilidade se essa comprovação for necessária.
Skill ou subagent explicitamente solicitado não é encontrado ou não pode ser utilizado	FAIL de wiring	Verifique caminho, frontmatter, nomes e referências; corrija a conexão e execute uma nova rodada controlada.
Há evidência de que a Skill executou, mas o Contract esperado não foi recuperado	FAIL de wiring	Verifique a referência relativa <code>contracts/report-request.contract.yaml</code> ; corrija-a e repita o teste.
Não é possível determinar se o Contract foi recuperado	INCONCLUSIVE de lineage	Não presuma falha nem sucesso; registre a limitação e melhore a observabilidade se necessário.
A resposta trata SignalR como fonte durável ou não oferece estado consultável	FAIL funcional	Identifique qual artefato deveria impedir esse comportamento, corrija sua origem e execute uma nova rodada.
A resposta parece adequada em parte, mas contradiz uma Rule ou pós-condição aplicável do Contract	FAIL funcional ou de consistência	Localize a obrigação violada e a transição responsável por preservá-la antes de alterar qualquer artefato.

A tabela orienta o diagnóstico, mas a correção deve preservar a relação entre **causa observada e artefato responsável**. Não ajuste manualmente a resposta produzida pelo Agent para fazê-la parecer correta.

Quando houver FAIL, siga esta ordem:

```

critério que falhou
  ↓
evidência observada
  ↓
transição relacionada
  ↓
artefato responsável
  ↓
correção
  ↓
nova execução

```

Por exemplo, se o resultado não oferecer um `statusUrl`, primeiro identifique qual expectativa foi violada e qual artefato deveria preservá-la. A correção pode estar na Rule, no Contract, na Skill ou no encadeamento entre eles; não escolha um arquivo apenas porque ele parece o local mais provável.

Depois da correção, **não sobrescreva a rodada anterior**. Preserve:

```

run-001-response.md
run-001.md

```

e registre a nova execução como:

```
run-002-response.md
run-002.md
```

Se a correção alterar apenas os artefatos ou o wiring, mantenha `expected.md` inalterado: o teste continua verificando a mesma expectativa.

Se descobrir que a própria expectativa estava incorreta ou incompleta, não altere silenciosamente o critério usado para avaliar uma execução já realizada. Preserve a rodada anterior, atualize ou versione a expectativa e só então execute uma nova rodada.

A decisão pode ser resumida assim:

```
PASS funcional + PASS lineage
→ avançar
```

```
PASS funcional + INCONCLUSIVE lineage
→ preservar limitação
→ melhorar observabilidade apenas se lineage for necessário
```

```
FAIL
→ localizar causa
→ corrigir origem
→ preservar Run 001
→ executar Run 002
```

```
expectativa incorreta
→ preservar avaliação anterior
→ revisar/versionar expected
→ executar nova rodada
```

Ao final desta etapa, o leitor deve saber **se pode avançar, se precisa corrigir o sistema ou se apenas não possui evidência suficiente para concluir**. A próxima etapa remove parte do controle explícito do primeiro teste para avaliar se o sistema consegue descobrir e rotear para os recursos adequados por conta própria.

1.1.11 10. Depois do wiring, teste a descoberta

Depois que a rodada controlada demonstrar que o caminho configurado funciona, execute uma segunda rodada removendo do prompt a referência explícita ao subagent. O objetivo agora é testar se o sistema consegue **descobrir e rotear para os recursos adequados a partir da própria tarefa**.

Mantenha os mesmos artefatos e o mesmo `expected.md`. Alterar outras variáveis nesta etapa dificultaria distinguir uma falha de discovery de uma mudança no próprio experimento.

Inicie **uma nova conversa no Agent do Cursor** e cole:

```
# Tarefa

## Objetivo

Propor o contrato para iniciar uma geração de relatório que pode durar minutos.

## Restrições

- Use os artefatos aplicáveis deste repositório.
- Não trate a resposta inicial como conclusão do processamento.
- Não trate notificação em tempo real como fonte durável do estado.
- Informe os artefatos que sustentaram a proposta.

## Saída esperada
```

1. Contrato HTTP mínimo.
2. Ordem das operações.
3. Fallback para reconexão do front-end.
4. Artefatos utilizados.

A diferença entre as duas rodadas é intencional:

RUN 001 – wiring controlado

prompt

- indica backend-development
- caminho conhecido é exercitado
- verifica se as conexões funcionam

RUN 002 – discovery

prompt

- descreve apenas a tarefa
- o sistema precisa reconhecer a aplicabilidade
- descobrir e rotear para os recursos adequados

Não altere os critérios funcionais entre as duas execuções. O que muda é apenas a forma como o caminho até os recursos é encontrado.

Ao final, preserve a nova resposta como:

```
.cursor/
├── evidence/
│   ├── checkpoints/
│   │   ├── contexto-minimo/
│   │   │   ├── run-001-response.md
│   │   │   ├── run-001.md
│   │   │   ├── run-002-response.md
│   │   └── run-002.md
```

Copie a resposta da segunda rodada para run-002-response.md e avalie-a em run-002.md usando o mesmo expected.md empregado na primeira execução.

Compare então as duas rodadas:

Pergunta	Run 001 — wiring	Run 002 — discovery
O resultado funcional atende ao esperado?	Registrar resultado	Registrar resultado
O subagent esperado participou?	Caminho solicitado explicitamente	Deve ser descoberto ou selecionado sem indicação explícita
A Skill participou?	Registrar evidência disponível	Registrar evidência disponível
O Contract foi alcançado pela Skill?	Registrar evidência disponível	Registrar evidência disponível
Há evidência suficiente para afirmar lineage?	PASS/FAIL/INCONCLUSIVE	PASS/FAIL/INCONCLUSIVE

A interpretação depende da diferença entre as rodadas.

Run 001 passa e Run 002 passa: o caminho funciona quando solicitado e também consegue ser descoberto a partir da tarefa. O checkpoint atingiu seu objetivo.

Run 001 passa e Run 002 falha: como a capacidade já funcionou no cenário controlado, investigue primeiro **discovery, descrições, sinais de aplicabilidade ou roteamento**, em

vez de concluir imediatamente que a Skill ou o Contract estão incorretos.

Run 001 falha: não use Run 002 para diagnosticar discovery. Corrija primeiro o wiring controlado, pois ainda não existe uma baseline confiável que demonstre que o caminho funciona quando explicitamente solicitado.

Run 002 produz resposta funcionalmente correta, mas o lineage permanece INCONCLUSIVE: preserve o resultado funcional, mas não afirme que o sistema descobriu determinado artefato sem evidência suficiente para sustentá-lo.

O experimento completo passa, portanto, de uma pergunta controlada para uma pergunta progressivamente mais difícil:

```
primeiro
"O caminho funciona quando eu o indico?"
      ↓
depois
"O sistema encontra esse caminho quando eu não o indico?"
```

Essa separação ajuda a localizar a origem de uma falha sem misturar **capacidade, wiring e discovery** no mesmo diagnóstico.

1.2 Do arquivo backend caótico à separação

O próximo exemplo é o ponto de partida da refatoração. Ao lê-lo, observe quais trechos **orientam decisões, coordenam etapas, oferecem conhecimento, descrevem capacidades especializadas, impõem restrições ou ajudam a verificar o resultado**.

A intenção não é criar um arquivo para cada frase, mas separar conteúdos que desempenham responsabilidades diferentes durante a execução.

O arquivo backend caótico. Considere um único documento com instruções como:

```
<!-- ANTI-PATTERN: instruções de responsabilidades diferentes no mesmo arquivo -->
```

```
Você é um desenvolvedor backend sênior.
Use Clean Architecture, DDD e SOLID.
Para operações demoradas, crie jobs, publique em fila e avise o frontend.
O SignalR permite atualização em tempo real.
Nunca notifique antes de persistir o resultado.
POST retorna 202 e GET consulta o status.
Use retry, idempotência, logs e traces.
Rode todos os testes e documente a solução.
```

O problema não é a ausência de informação. O documento reúne conteúdo potencialmente útil, mas mistura responsabilidades com aplicabilidade, autoridade e ciclos de uso diferentes:

- papel e orientação do agente;
- princípios arquiteturais;
- coordenação da operação assíncrona;
- conhecimento sobre comunicação em tempo real;
- contrato HTTP;
- obrigação transacional;
- práticas de resiliência e observabilidade;
- execução de testes e registro da conclusão.

O primeiro passo da separação é identificar **qual responsabilidade cada trecho parece exercer**. Como algumas frases ainda não informam autoridade ou aplicabilidade suficientes, a tabela mostra destinos prováveis, e não uma conversão automática:

Conteúdo encontrado	Destino provável	Motivo
Você é um desenvolvedor backend sênior.	Agent	Define papel e orientação recorrente.
Use Clean Architecture, DDD e SOLID.	Agent, Rule ou Knowledge, conforme autoridade e uso	Pode representar orientação recorrente, obrigação arquitetural ou conteúdo consultivo; a frase isolada não permite decidir.
Criar job, publicar em fila e avisar o frontend em operações demoradas	Playbook	Coordena etapas e dependências para atingir um objetivo.
O SignalR permite atualização em tempo real.	Knowledge	Explica uma capacidade tecnológica sem, por si só, impor uma obrigação.
Nunca notifique antes de persistir o resultado.	Rule; o Contract pode refletir a pós-condição aplicável	A Rule preserva a obrigação normativa; o Contract pode tornar a condição esperada verificável sem substituir sua fonte de autoridade.
POST retorna 202 e GET consulta o status.	Contract	Define comportamento observável da interface HTTP.
Retry e idempotência	Rule, Playbook ou Knowledge, conforme autoridade e uso	Podem representar obrigação, etapa operacional ou orientação consultiva.
Logs e traces	Rule ou Playbook; os dados produzidos podem sustentar Evidence	A instrução pode exigir instrumentação ou coordenar sua produção; a telemetria observada pode depois servir como evidência, mas não é automaticamente um Evidence Record.
Rode todos os testes.	Playbook; critérios verificáveis pertencem à Eval	Executar testes é uma etapa operacional; definir o que constitui sucesso ou falha exige expectativas explícitas.
Documente a solução.	Evidence ou documentação, conforme o conteúdo registrado	Só constitui Evidence quando preserva fatos observados e rastreáveis da execução ou avaliação.

A tabela evidencia por que a separação não pode ser feita apenas pelo vocabulário. Verbos como “use”, “rode” ou “documente” indicam uma ação, mas não revelam sozinhos **quem possui autoridade, quando a instrução se aplica ou qual artefato deve governá-la**.

Por isso, a refatoração não consiste em mover frases mecanicamente para novas pastas. Ela precisa tornar explícitos **responsabilidade, autoridade, aplicabilidade e mecanismo de uso** antes de decidir onde cada conteúdo deve permanecer.

1.3 Agent: o conteúdo HOT

Na separação do arquivo backend caótico, comece pelo conteúdo que precisa orientar uma parcela ampla das tarefas. Na ontologia deste livro, esse é o papel do **Agent**: concentrar missão, invariantes, roteamento, condições de parada e evidência mínima de conclusão sem absorver procedimentos, conhecimento detalhado ou contratos que possuem ciclos de uso próprios.

O Agent recebe o pedido, identifica o próximo recurso aplicável e decide quando a execução pode continuar, precisa de outro artefato ou deve ser interrompida por falta de informação suficiente.

Antes do exemplo, observe cinco elementos:

1. **missão:** delimita o resultado que o Agent coordena;
2. **invariantes:** preservam condições que devem permanecer verdadeiras;
3. **roteamento:** indica como reconhecer o próximo recurso aplicável;
4. **condições de parada:** determinam quando não é seguro continuar;
5. **evidência mínima:** define o que precisa existir antes de considerar a tarefa concluída.

i Agent conceitual e adapter do Cursor

O Agent Card é uma estrutura da ontologia deste livro. No Cursor, sua responsabilidade é materializada por um adapter compatível com o mecanismo de subagents do host. Metadados utilizados pela ontologia do livro não devem ser tratados automaticamente como campos nativos do Cursor.

BOOK ONTOLOGY — Agent. CURSOR NATIVE — adapter. No laboratório, o adapter fica em `.cursor/agents/backend-development.md` e transporta para o host as instruções necessárias à execução:

```
---
name: backend-development
description: Coordena mudanças backend pequenas, seguras e verificáveis.
---

# Backend Development

## Missão

Implementar mudanças backend pequenas, seguras e verificáveis.

## Invariantes

- Mudanças HTTP exigem validação de contrato.
- Operações mutáveis exigem política de idempotência.
- Repetir a mesma solicitação não deve criar efeito lógico duplicado.
- Não invente dependências, endpoints ou regras ausentes.

## Roteamento

- Quando a tarefa exigir processamento assíncrono, utilize o playbook
  `.cursor/playbooks/operacao-assincrona.md`.
- Quando a tarefa envolver criação, alteração ou validação de contrato HTTP, utilize a skill
  `.cursor/skills/validate-api-contract/SKILL.md`.
- Para persistência ou mensageria, utilize apenas capacidades existentes no repositório quando
  houver sinal claro de aplicabilidade.

## Condições de parada

- Interrompa e solicite a informação necessária quando contrato ou critério de aceite estiver
  ausente e não puder ser recuperado das fontes disponíveis.

## Evidência mínima

- Conclua somente quando os critérios aplicáveis tiverem sido verificados e houver evidência
  suficiente para sustentar a conclusão.
```

O exemplo mostra que HOT não significa apenas **poucas linhas**. O conteúdo precisa concentrar o que é necessário para manter orientação recorrente sem trazer para o contexto todos os detalhes

da execução.

Um Agent HOT deve permitir reconhecer:

1. qual responsabilidade está sendo coordenada;
2. quais invariantes não podem ser violadas;
3. quais sinais levam a outros recursos;
4. quando a execução precisa parar;
5. o que deve existir antes da conclusão.

Uma frase como “use as skills quando necessário” é curta, mas insuficiente: ela não informa **como reconhecer quando uma Skill se aplica** e transfere a decisão inteira para inferência do modelo.

1.3.1 Teste mínimo do Agent HOT

Valide duas responsabilidades do Agent: **roteamento** e **condição de parada**.

No primeiro cenário, forneça uma tarefa de alteração de contrato HTTP com informações suficientes para prosseguir e verifique se o Agent reconhece a aplicabilidade da skill `validate-api-contract`.

No segundo, forneça uma tarefa que dependa de um critério de aceite ausente e não recuperável pelas fontes disponíveis. O comportamento esperado é solicitar a informação necessária ou interromper a execução, sem inventar o critério.

Os dois cenários reutilizam o método já apresentado no capítulo:

```
expectativa
  ↓
execução
  ↓
resultado observado
  ↓
evidência
  ↓
interpretação
```

O objetivo deste teste é verificar se o conteúdo HOT fornece informação suficiente para **rotear quando existe um próximo recurso aplicável e parar quando não existe fundamento suficiente para continuar**.

O procedimento reproduzível — incluindo `expected.md`, prompts das duas rodadas, arquivos `run-XXX`, critérios de PASS, FAIL e INCONCLUSIVE e interpretação dos resultados — está disponível em [Laboratório — Agent HOT](#).

1.4 Playbook: coordenação WARM

Depois de separar o conteúdo HOT do Agent, o próximo passo é retirar dele os procedimentos que só precisam entrar no contexto quando uma tarefa específica os torna aplicáveis. Na ontologia deste livro, essa é uma das responsabilidades do **Playbook**.

O Playbook é produzido por quem conhece a sequência operacional, suas dependências e condições de interrupção. Ele é utilizado depois que o Agent reconhece que determinado tipo de tarefa exige aquele procedimento. Seu papel não é definir a missão geral nem executar sozinho uma capacidade especializada, mas **coordenar os recursos necessários para alcançar um resultado**.

Ao ler o exemplo, acompanhe quatro elementos:

1. **sinais de aplicabilidade:** indicam quando o procedimento deve ser considerado;
2. **pré-condições:** estabelecem o que precisa existir antes da execução;
3. **fluxo:** organiza a sequência e as dependências;
4. **condições de parada:** determinam quando o procedimento não pode prosseguir com

segurança.

Essa estrutura ajuda a distinguir um procedimento operacional de uma lista genérica de boas práticas.

BOOK ONTOLOGY — convenção do repositório. O Cursor não atribui comportamento especial à pasta `.cursor/playbooks/` apenas por sua existência. Neste modelo, um Playbook participa da execução quando algum mecanismo reconhecido pelo host — como um Agent, Skill, Rule, script ou harness — referencia ou recupera explicitamente seu conteúdo.

```

---
id: PLAYBOOK-ASYNC-001
name: operacao-assincrona
type: playbook
description: Coordena operações longas executadas por job e reconciliadas por API.
temperature: warm
applicability_signals:
  - processamento excede o tempo síncrono esperado
  - conclusão precisa atualizar o front-end
owner: backend-platform
---

# Operação assíncrona

## Quando usar

Usar quando a operação não puder concluir dentro do tempo síncrono esperado.

## Pré-condições

- Contrato de entrada definido.
- Identidade e autorização disponíveis.
- Política de idempotência definida.
- Estados do job conhecidos.

## Fluxo

1. Validar a requisição utilizando `cursor/skills/validate-api-contract/SKILL.md` quando houver contrato HTTP aplicável.
2. Criar o job durável.
3. Publicar a mensagem.
4. Processar com correlação.
5. Persistir o resultado.
6. Confirmar a transação.
7. Notificar o frontend.
8. Preservar reconciliação via API.

## Condições de parada

- Contrato ausente.
- Estado final não persistente.
- Resultado da tool não verificável.
- Notificação sem identificador correlacionável.

```

O exemplo mostra por que o Playbook é WARM: a sequência possui valor quando a tarefa apresenta sinais que justificam sua recuperação, mas não precisa ocupar o contexto de tarefas que não executam esse fluxo.

Durante a coordenação, o Playbook pode:

- encaminhar etapas para Skills;
- indicar Knowledge aplicável;
- exigir o cumprimento de Rules;
- utilizar Contracts para verificar condições esperadas;
- determinar quando a execução deve ser interrompida.

Isso não transforma o Playbook no proprietário desses outros artefatos. Sua responsabilidade continua sendo **orquestrar a sequência**, enquanto cada recurso preserva seu próprio papel.

1.4.1 Teste mínimo do Playbook WARM

Como o Playbook é WARM, sua recuperação deve depender da **aplicabilidade do procedimento**, e não apenas da presença de palavras relacionadas ao domínio. O teste mínimo verifica exatamente essa fronteira usando dois cenários opostos.

Valide dois comportamentos do playbook `.cursor/playbooks/operacao-assincrona.md`:

- **cenário aplicável:** em uma tarefa cujo processamento realmente continue depois da resposta síncrona, verifique se o playbook `operacao-assincrona` é recuperado e utilizado;
- **cenário não aplicável:** em uma tarefa que possa terminar dentro da própria requisição, verifique se o Playbook permanece fora do fluxo, mesmo que o pedido contenha termos genéricos como “API”, “processamento” ou “backend”.

A comparação permite verificar se os sinais definidos em `applicability_signals` diferenciam **necessidade real do procedimento** de mera semelhança lexical:

```
processamento realmente assíncrono
    ↓
Playbook aplicável
    ↓
recuperar e utilizar

processamento síncrono
    ↓
Playbook não aplicável
    ↓
não recuperar desnecessariamente
```

O procedimento reproduzível — incluindo expectativa registrada antes da execução, prompts dos dois cenários, coleta das respostas, Evidence Records e critérios de PASS, FAIL e INCONCLUSIVE — está disponível em **Laboratório — Playbook WARM**.

Playbook e localização física

Neste modelo, `.cursor/playbooks/` é uma convenção de organização para procedimentos coordenadores. Outro host ou repositório pode armazená-los em outro local sem alterar sua responsabilidade semântica, desde que exista um mecanismo capaz de referenciá-los ou recuperá-los quando aplicáveis.

1.5 Skill: capacidade especializada sob demanda

Quando o Agent ou um Playbook identifica uma atividade especializada aplicável, a Skill fornece instruções e recursos para executá-la. Ela encapsula uma capacidade reutilizável e delimitada sem assumir a coordenação completa da tarefa.

No modelo deste livro, uma Skill deve permitir observar o resultado da capacidade executada, como um relatório de compatibilidade, um código de saída ou uma incompatibilidade encontrada.

OPEN SPEC + CURSOR NATIVE — Agent Skill. O arquivo fica em `.cursor/skills/validate-api-contract/SKILL.md` e segue o formato Agent Skills suportado pelo Cursor.

```

validate-api-contract/
├── SKILL.md
├── contracts/
│   └── report-request.contract.yaml
├── references/
│   └── http-contract-guidance.md
├── scripts/
│   └── validate-openapi.py
└── assets/
    └── contract-report-template.yaml

```

O diretório `contracts/` é uma extensão da ontologia deste livro para manter junto da Skill um Contract de uso exclusivo. O Agent Skills permite recursos adicionais dentro do pacote, mas não define `contracts/` como um diretório especial.

```

---
name: validate-api-contract
description: >
  Use esta skill para analisar ou validar mudanças em contratos HTTP,
  OpenAPI, status codes, headers e compatibilidade de clientes.
metadata:
  display-name: Validar Contrato de API
  owner: backend-platform
  book-temperature: warm
  book-authority: advisory
---

# Validar contrato de API

## Quando usar

- Alteração de contrato HTTP, OpenAPI, status code, header ou schema.

## Procedimento

1. Identifique a operação e os consumidores.
2. Leia `contracts/report-request.contract.yaml` quando a operação analisada estiver coberta por
   esse Contract.
3. Compare entrada, saída, erros, headers e pós-condições aplicáveis.
4. Consulte `references/http-contract-guidance.md` quando precisar de orientação adicional.
5. Execute `scripts/validate-openapi.py` quando houver uma especificação OpenAPI disponível para
   validação.
6. Registre incompatibilidades e evidências usando `assets/contract-report-template.yaml` quando
   aplicável.

## Saída e evidência

- Relatório de compatibilidade vinculado ao contrato analisado.
- Comando, código de saída e resultado do validador registrados quando o script for executado.

```

Os campos `book-temperature` e `book-authority` pertencem à ontologia deste livro. Eles ficam dentro de `metadata`, que o Agent Skills permite estender com propriedades adicionais, mas não devem ser interpretados como comportamento nativo da especificação.

A Skill também demonstra progressive disclosure dentro do próprio pacote:

```

name + description
↓
Skill considerada aplicável

```

↓
SKILL.md
↓
recurso necessário

A *description* participa da descoberta porque ajuda o agente a decidir se a Skill é relevante para a tarefa. Quando a Skill é selecionada, o Agent Skills denomina a próxima etapa de **activation**: o conteúdo completo de SKILL.md é carregado para orientar a execução. Os recursos adicionais são recuperados somente quando necessários.

Por isso, uma descrição ampla demais pode fazer a Skill ser considerada em tarefas irrelevantes, enquanto uma descrição estreita demais pode impedir sua seleção em formulações válidas.

1.5.1 Teste mínimo da Skill

Avalie a descrição com casos positivos, paráfrases e casos negativos:

Caso	Prompt	Resultado esperado
should-trigger	“Altere o schema de resposta do endpoint e valide o OpenAPI.”	A Skill deve ser considerada aplicável.
Paráfrase	“Esse novo campo quebra clientes antigos?”	A Skill deve ser considerada aplicável.
should-not-trigger	“Explique a regra de retry do worker.”	A Skill não deve ser selecionada apenas por esse pedido.

Uma descrição fraca seria “ajuda com backend”. Uma descrição melhor explicita tanto **o que a Skill faz** quanto **quando ela se aplica**: “analisa ou valida mudanças em contratos HTTP, OpenAPI, status codes, headers e compatibilidade de clientes”.

Como a decisão de seleção envolve o agente e pode variar entre execuções, um único resultado não deve ser tratado como garantia determinística. Use diferentes formulações e preserve casos de validação que não tenham sido utilizados para ajustar a descrição.

O procedimento reproduzível, com conjunto positivo e negativo, múltiplas execuções, registro das ativações observadas e critérios de avaliação, está disponível em [Laboratório — Skill e descoberta](#).

★ Playbook não é Skill

Um Playbook coordena várias capacidades para atingir um objetivo. Uma Skill encapsula uma capacidade especializada e reutilizável.

A presença de *references/*, *scripts/*, *assets/* ou outros recursos não determina sozinha a responsabilidade do pacote. Em Agent Skills, o pacote precisa conter SKILL.md com *name*, *description* e instruções que definam sua capacidade.

1.6 Knowledge: fundamentação WARM ou COLD

Depois que Agent, Playbook e Skill definem **como a execução deve avançar**, ainda podem existir decisões que dependem de conhecimento especializado sobre domínio, arquitetura ou tecnologia. Na ontologia deste livro, esse é o papel de **Knowledge**.

Knowledge é curado por quem mantém o conhecimento descrito. Agent, Playbook ou Skill pode recuperá-lo quando precisa compreender um conceito, interpretar uma situação ou sustentar uma decisão. Seu conteúdo não coordena a execução e não cria obrigação normativa por conta própria.

No exemplo, o conhecimento sobre SignalR ajuda a interpretar **o papel da tecnologia e seus limites**. A existência do arquivo, porém, não demonstra que ele participou de uma execução:

quando essa rastreabilidade for necessária, deve existir evidência suficiente de que o conteúdo aplicável foi recuperado e utilizado na decisão.

Knowledge pode conter:

- limites e conceitos do domínio;
- mapas e explicações da arquitetura;
- schemas e modelos de dados;
- documentação de APIs;
- comportamento de mensageria;
- convenções de observabilidade;
- catálogos de falhas;
- decisões históricas e referências técnicas.

BOOK ONTOLOGY — convenção do repositório. Neste modelo, os conteúdos compartilhados de Knowledge permanecem em `.cursor/knowledge/` e precisam ser referenciados ou recuperados por algum mecanismo que participe da execução. A presença do arquivo nessa pasta não o coloca automaticamente no contexto.

Para este exemplo, utilizaremos:

```
.cursor/
└─ knowledge/
   └─ signalr-operacoes-assincronas.md
```

```
---
id: KNOWLEDGE-SIGNALR-001
name: signalr-operacoes-assincronas
type: knowledge
description: Explica o papel do SignalR em operações assíncronas.
temperature: warm
authority: advisory
owner: backend-platform
---

# SignalR em operações assíncronas

## Use para

- compreender atualização em tempo real;
- comparar push e consulta por API;
- reconhecer limites de entrega.

## Limite essencial

SignalR reduz a latência percebida, mas não substitui o estado durável.

## Fontes

- documentação oficial do ASP.NET Core SignalR;
- contrato interno da operação assíncrona.
```

O exemplo é WARM porque pode ser recuperado quando uma tarefa envolve comunicação em tempo real ou reconciliação de estado, mas não precisa acompanhar todas as tarefas backend. Um material consultado apenas em investigações, incidentes ou decisões históricas poderia permanecer COLD sem perder importância ou autoridade.

Ex Como reconhecer Knowledge

Se o conteúdo explica um domínio, tecnologia, decisão histórica ou alternativa e ajuda outro artefato a interpretar uma situação sem coordenar etapas ou impor sozinho uma obrigação, ele tende a desempenhar o papel de Knowledge.

Knowledge também não deve duplicar integralmente outros artefatos:

Agent

→ decide e roteia

Playbook

→ coordena etapas

Skill

→ executa uma capacidade especializada

Knowledge

→ apoia compreensão e decisão

1.6.1 Teste mínimo do Knowledge

Valide duas situações opostas:

- em uma tarefa que exija decidir entre notificação em tempo real e recuperação de estado por API, verifique se `.cursor/knowledge/signalr-operacoes-assincronas.md` é recuperado quando necessário e se a decisão permanece compatível com o limite documentado;
- em uma tarefa sem relação com comunicação em tempo real, verifique se esse Knowledge não é recuperado apenas por pertencer ao mesmo domínio backend.

O teste procura verificar **aplicabilidade e uso**, não apenas a existência do arquivo. Uma resposta tecnicamente correta, isoladamente, também não comprova que o Knowledge participou da execução.

O procedimento reproduzível, com cenário aplicável e não aplicável, expectativa prévia, coleta das respostas, evidência de recuperação e critérios de PASS, FAIL e INCONCLUSIVE, está disponível em **Laboratório — Knowledge WARM**.

Checkpoint: responsabilidades separadas

Até aqui, o Agent decide e roteia, o Playbook coordena, a Skill executa uma capacidade especializada e Knowledge apoia compreensão e decisão.

Se houver um problema de **roteamento**, investigue primeiro o Agent; se a **ordem das etapas** estiver incorreta, examine o Playbook; se a **execução da capacidade** estiver incompleta, examine a Skill; se uma **interpretação técnica estiver desatualizada ou incorreta**, investigue o Knowledge que a sustenta.

1.7 Anatomia dos pacotes

Agora que cada responsabilidade possui um papel, falta organizar os recursos internos sem carregá-los todos antecipadamente. Dentro de uma Agent Skill, essa organização favorece progressive disclosure: o `SKILL.md` mantém as instruções principais e aponta para recursos adicionais que podem ser recuperados ou executados somente quando necessários.

OPEN SPEC — Agent Skills. `references/`, `scripts/` e `assets/` são convenções previstas pela especificação para organizar recursos opcionais dentro de uma Skill. Neste livro, a mesma separação entre **instrução, consulta, execução e recurso reutilizável** também pode orientar outros pacotes, sem transformá-los automaticamente em Skills.

- `references/` — **conteúdo consultado:** documentação técnica, políticas, tabelas de

compatibilidade, catálogos de erros e outras informações que podem ser recuperadas quando necessárias para compreender ou executar a tarefa;

- **scripts/ — execução determinística:** validadores, comparadores, linters, geradores e outras rotinas executáveis. Quando sua execução fizer parte da evidência, registre comando, parâmetros, código de saída e resultado;
- **assets/ — recursos reutilizáveis:** templates, exemplos-base, schemas, arquivos de configuração e outros materiais utilizados como entrada ou base para produzir uma saída. Um Asset não deve ser interpretado automaticamente como instrução normativa;
- **contracts/ — Contract local:** convenção da ontologia deste livro para manter junto da capacidade um Contract utilizado exclusivamente por ela. Não é um diretório especial definido pelo Agent Skills.

A Skill `validate-api-contract` fica organizada assim:

```
validate-api-contract/
├── SKILL.md
├── contracts/
│   └── report-request.contract.yaml
├── references/
│   └── http-contract-guidance.md
├── scripts/
│   └── validate-openapi.py
└── assets/
    └── contract-report-template.yaml
```

Esses diretórios não formam um checklist obrigatório. A Skill deve conter apenas os recursos necessários para sua responsabilidade, mantendo-os próximos de quem realmente os utiliza.

A partir de `SKILL.md`, referências aos recursos internos devem usar caminhos relativos ao pacote, por exemplo:

```
contracts/report-request.contract.yaml
references/http-contract-guidance.md
scripts/validate-openapi.py
assets/contract-report-template.yaml
```

Isso preserva a Skill como uma unidade mais portátil e evita acoplamento desnecessário à localização global do repositório. A especificação Agent Skills recomenda referências relativas a partir da raiz da Skill.

★ Cada recurso possui uma função

Reference é consultada para fornecer informação adicional. Script é executado. Asset é reutilizado como recurso. Neste livro, um Contract local fornece critérios e restrições verificáveis para a capacidade que o possui.

Separar essas funções ajuda a evitar que todo conteúdo auxiliar seja carregado ou interpretado da mesma forma.

1.8 YAML frontmatter: descoberta, roteamento e governança

Frontmatter é o bloco YAML delimitado por `---` no topo de um artefato. Ele separa metadados do conteúdo principal e permite que mecanismos de catálogo, descoberta ou roteamento inspecionem informações sobre o artefato sem depender da leitura integral do corpo.

Neste livro, use-o para:

- descrever o artefato e sua aplicabilidade;
- filtrar por tipo, domínio, estado ou autoridade;

- registrar sinais de aplicabilidade antes de recuperar o procedimento completo;
- relacionar Rules, Contracts, Playbooks e Evals;
- registrar proprietário, versão e revisão;
- manter metadados operacionais próximos ao início do artefato.

A presença desses campos não cria comportamento por conta própria. Algum consumidor — host, Agent, Skill, script, índice ou harness — precisa interpretá-los para que participem de descoberta, roteamento ou governança.

! Frontmatter não garante cache

Metadados estáveis podem favorecer reutilização de prefixos quando o runtime e o provedor oferecem prompt caching, mas o YAML não cria cache por si só. O efeito depende, entre outros fatores, da serialização, posição do conteúdo, política do provedor e frequência de reutilização.

Meça o resultado no ambiente real; não trate economia de tokens ou latência como propriedade automática do frontmatter.

1.8.1 id e name resolvem problemas diferentes

Na ontologia deste livro, artefatos reutilizáveis e catalogados podem possuir id e name, mas os dois campos não devem ser tratados como equivalentes:

Campo	Responsabilidade	Regra prática
id	Identidade técnica estável para relacionamentos, auditoria, histórico e migração.	Evite alterá-lo apenas porque nome ou título foram revisados.
name	Identificador operacional do artefato, utilizado em catálogo, descoberta, roteamento ou referência.	Use kebab-case em todos os artefatos do livro e trate mudanças como potencialmente incompatíveis com consumidores existentes.
description	Explica o que o artefato faz e, quando aplicável, em quais situações deve ser considerado.	Deve fornecer informação suficiente para reconhecer sua aplicabilidade sem depender da leitura integral do corpo.

O título destinado ao leitor é uma dimensão diferente do name. Por exemplo:

```
id: PLAYBOOK-ASYNC-001
name: operacao-assincrona
```

pode ser apresentado no corpo como:

```
# Operação assíncrona
```

Agent, Playbook, Knowledge, Rule, Contract e Eval podem utilizar id e name segundo essa convenção da ontologia do livro. Isso não significa, porém, que todas as especificações ou hosts reconheçam esses campos da mesma maneira.

Um Evidence Record de execução é diferente: ele representa uma instância produzida durante uma execução e deve possuir uma identidade própria, como evidenceId. O schema ou tipo de Evidence pode ser um artefato reutilizável com id e name; cada registro produzido não precisa receber um nome decorativo.

O campo ``name`` de uma Agent Skill possui um contrato próprio. No frontmatter de ``SKILL.md``, ele é obrigatório e participa da identificação da Skill.

Segundo a especificação Agent Skills, ``name`` deve:

- possuir de 1 a 64 caracteres;
- conter apenas letras minúsculas, números e hífens;
- não começar nem terminar com hífen;
- não possuir hífens consecutivos;
- coincidir com o nome do diretório da Skill.

Por exemplo:

```
```text
validate-api-contract/
└─ SKILL.md
```
```

deve conter:

```
```yaml

name: validate-api-contract
description: Analisa ou valida mudanças em contratos HTTP, OpenAPI, status codes, headers e
compatibilidade de clientes.

```
```

O título destinado à leitura humana pode permanecer no corpo:

```
```markdown
Validar contrato de API
```
```

ou em `metadata`, quando algum catálogo ou consumidor realmente utilizar essa informação.

```
::: {.warning title="name não é nome de apresentação"}
```

Em Agent Skills, `name` identifica a Skill e deve corresponder ao diretório.

Use `kebab-case` no frontmatter e mantenha o título humano separado.

```
:::
```

A distinção é importante porque **frontmatter é estrutura de metadados; descoberta, roteamento, autoridade e cache são comportamentos dos mecanismos que o consomem**. Não atribua ao YAML capacidades que pertencem ao host ou ao runtime.

1.9 Frontmatter de uma Skill

Na especificação Agent Skills, `name` e `description` são obrigatórios e participam da descoberta. Campos próprios da organização devem permanecer dentro de `metadata`, a menos que o cliente utilizado documente outra extensão.

Observe no exemplo a divisão entre contrato interoperável e governança interna: `name` e `description` ajudam o cliente a descobrir a Skill; os campos dentro de `metadata` só produzem efeito quando algum catálogo, host ou runtime da organização os interpreta.

```
---
name: validate-api-contract
description: >
```

Use para validar contratos HTTP, OpenAPI, status codes, headers e compatibilidade de clientes.

metadata:

*display-name: Validar Contrato de API
owner: backend-platform
book-temperature: warm
book-authority: advisory*

Validar Contrato de API

Quando usar

- Mudança de endpoint, schema, status code ou header.

Os campos book-temperature e book-authority pertencem à ontologia deste livro. O prefixo ajuda a distingui-los dos campos definidos pela especificação Agent Skills ou pelo host.

1.9.1 Frontmatter de um Playbook

No Playbook, os metadados precisam tornar sua responsabilidade e os sinais de aplicabilidade reconhecíveis antes da leitura do procedimento completo. Neste livro, id, type, temperature, applicability_signals, owner, status e version pertencem à convenção de governança do repositório; algum mecanismo conectado à execução continua responsável por referenciar ou recuperar o Playbook quando aplicável.

id: PLAYBOOK-ASYNC-001

name: operacao-assincrona

type: playbook

description: Coordena operações longas executadas por job.

temperature: warm

applicability_signals:

- operação excede o tempo síncrono esperado
- conclusão precisa ser reconciliável

owner: backend-platform

status: approved

version: 2.1.0

Operação assíncrona

Quando usar

- A operação excede o tempo síncrono esperado.
- A conclusão precisa ser reconciliável pela API.

O nome permanece em kebab-case, enquanto o título apresentado ao leitor pode usar linguagem natural.

1.9.2 Frontmatter de uma Rule

Na Rule, o ponto central é a obrigação que precisa ser preservada durante tarefas aplicáveis. No Cursor, porém, a participação de uma Project Rule na execução não é controlada por campos como authority ou applies_to, mas pelo mecanismo nativo de Rules do host.

Para uma Rule recuperada de acordo com a relevância da tarefa, utilize o modo **Agent Requested**: description informa ao Agent quando a Rule é útil, enquanto alwaysApply: false evita que ela seja carregada em todas as solicitações.

Neste exemplo, a Rule fica em:

```
.cursor/
├─ rules/
│   └─ persistir-antes-de-notificar.mdc
```

```
---
description: >
  Aplicar em operações assíncronas que persistem um estado final
  e depois notificam o cliente sobre a conclusão.
globs:
alwaysApply: false
---

# Persistir antes de notificar

## Obrigação

- Persistir e confirmar o estado final antes de publicar a notificação.
- A notificação não substitui o estado durável como fonte de reconciliação.
```

Nesse formato, a description fornece o **sinal de aplicabilidade** que permite ao Agent considerar a Rule quando a tarefa envolve persistência seguida de notificação.

A autoridade normativa pertence ao conteúdo e à governança definida pelo projeto; ela não é criada por um campo authority reconhecido pelo Cursor.

Rule do livro e Project Rule do Cursor

Na ontologia deste livro, a mesma Rule pode possuir informações de governança como id, name, authority, owner, status e version em um catálogo ou fonte canônica própria.

O adapter `.cursor/rules/persistir-antes-de-notificar.mdc`, porém, deve utilizar o contrato reconhecido pelo Cursor para determinar quando a Rule entra no contexto.

Compare os três exemplos antes de avançar:

| Artefato | Informação usada para descoberta | Informação que delimita aplicabilidade | Consumidor esperado |
|----------|----------------------------------|---|--|
| Skill | name e description | description e instruções do pacote | Cliente compatível com Agent Skills |
| Playbook | description | applicability_signals | Agent, Skill ou outro mecanismo que referencie ou recupere o arquivo |
| Rule | description | applies_to e mecanismo de integração com o host | Governança do projeto e, quando conectada, Cursor |

A tabela evidencia uma diferença importante: os mesmos campos não possuem necessariamente a mesma semântica operacional em todos os artefatos. É o consumidor que determina como esses metadados participam de descoberta, roteamento ou aplicação.

Evite campos voláteis no conteúdo carregado com frequência, como timestamps de cada execução. Eles reduzem estabilidade, dificultam comparação entre versões e podem diminuir oportunidades de reaproveitamento de prefixos quando o provedor oferece prompt caching.

★ **Aprofundamento no capítulo de templates**

A anatomia ampliada e o exemplo completo de Kubernetes estão no Laboratório 3 deste workbook. Aqui, preserve apenas a decisão essencial: uma Skill precisa declarar escopo, procedimento, saída e evidência; seções adicionais só devem existir quando melhorarem a execução ou sua verificabilidade.

Capítulo 2

Rules: obrigação, proibição e fallback

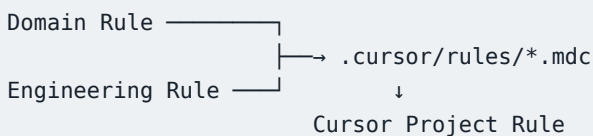
Uma Rule torna explícito um comportamento que não deve depender apenas da interpretação do Agent. Ela pode declarar uma obrigação, proibir um atalho conhecido, definir um fallback e indicar quais evidências precisam ser preservadas.

Neste livro, todas as Rules executáveis seguem **um único mecanismo técnico**: são Cursor Project Rules armazenadas em `.cursor/rules/` e utilizam o formato reconhecido pelo host.

A diferença entre elas é **semântica**, isto é, depende da origem e do significado da obrigação:

- **Domain Rule:** expressa uma obrigação proveniente do domínio, negócio ou política que o sistema precisa preservar;
- **Engineering Rule:** expressa uma obrigação técnica ou arquitetural necessária para segurança, consistência, confiabilidade ou qualidade da solução.

As duas continuam sendo Project Rules para o Cursor:



Essa separação permite distinguir **por que a regra existe** sem criar dois formatos ou mecanismos de execução diferentes.

i Semântica e aplicação são dimensões diferentes

Domain Rule e Engineering Rule classificam o **significado da obrigação**.

Always, Auto Attached, Agent Requested e Manual descrevem **como uma Project Rule participa do contexto no Cursor**.

Uma Domain Rule pode, por exemplo, ser Agent Requested; uma Engineering Rule pode ser Always. Uma dimensão não determina automaticamente a outra.

2.1 Engineering Rule: persistir antes de notificar

A obrigação usada neste exemplo é arquitetural: uma operação assíncrona não deve comunicar sua conclusão antes que o estado final esteja persistido e confirmado.

Por isso, ela é classificada semanticamente como **Engineering Rule** e materializada diretamente como uma Cursor Project Rule:

```
.cursor/
├── rules/
│   └── persistir-antes-de-notificar.mdc
```

```

---
description: >
  Aplicar em operações assíncronas que persistem um estado final
  e notificam o cliente sobre a conclusão.
globs:
alwaysApply: false
---

# Persistir antes de notificar

**Classificação semântica:** Engineering Rule

## Obrigação

- DEVE persistir o resultado final.
- DEVE confirmar a transação antes da publicação.

## Proibição

- NÃO DEVE tratar SignalR como fonte durável do estado.
- NÃO DEVE notificar conclusão antes da confirmação da persistência.

## Fallback

- SE a notificação falhar, preserve o estado final consultável pela API.

## Evidência

- REGISTRE `jobId`, `correlationId`, estado persistido e resultado da publicação.

```

Neste exemplo, a Rule utiliza o modo **Agent Requested**. A description fornece informação para que o Agent reconheça sua relevância, `alwaysApply: false` impede aplicação indiscriminada e `globs` permanece vazio porque a aplicabilidade depende da **semântica da operação**, não de um conjunto específico de arquivos.

Se a aplicabilidade dependesse dos arquivos envolvidos, poderiam ser utilizados `globs`. Se a obrigação precisasse estar disponível em todas as conversas do projeto, a configuração poderia utilizar `alwaysApply: true`.

A classificação `Engineering Rule` permanece no corpo porque ela pertence à organização semântica deste livro e não ao contrato de frontmatter interpretado pelo Cursor.

2.1.1 E uma Domain Rule?

Uma Domain Rule utiliza exatamente o mesmo formato técnico. O que muda é a origem da obrigação.

Por exemplo, uma política de domínio poderia estabelecer que determinado relatório só pode ser disponibilizado a uma identidade autorizada. Essa obrigação poderia ser materializada em:

```
.cursor/rules/relatorio-exige-autorizacao.mdc
```

com o mesmo frontmatter de Project Rules e a classificação semântica correspondente no corpo:

```

---
description: >
  Aplicar ao gerar ou disponibilizar relatórios cujo acesso
  dependa da identidade e autorização do solicitante.
globs:
alwaysApply: false

```

```
---
```

```
# Relatório exige autorização
```

```
**Classificação semântica:** Domain Rule
```

```
## Obrigação
```

```
- DEVE validar a autorização do solicitante antes de disponibilizar o relatório.
```

```
## Proibição
```

```
- NÃO DEVE disponibilizar o relatório quando a autorização necessária não puder ser comprovada.
```

Portanto, o leitor precisa aprender apenas um mecanismo técnico:

```
.cursor/rules/*.mdc
```

```
↓
```

```
Cursor Project Rules
```

e pode classificar semanticamente as obrigações conforme sua origem:

```
obrigação de domínio
```

```
→ Domain Rule
```

```
obrigação técnica ou arquitetural
```

```
→ Engineering Rule
```

Uma Rule normativa e reutilizável não deve permanecer escondida apenas dentro de um exemplo histórico, Knowledge ou Playbook. Quando a obrigação precisa participar diretamente da execução no Cursor, ela deve ser materializada como uma Project Rule e configurada de acordo com seu sinal real de aplicabilidade.

Como aplicar na prática: identifique a origem da obrigação; classifique-a como Domain Rule ou Engineering Rule; declare o comportamento esperado; acrescente a proibição de atalhos conhecidos, fallback e evidência quando necessários; escolha o modo de aplicação do Cursor de acordo com a forma como a Rule deve participar das tarefas.

2.1.2 Evitando drift entre a Rule e os artefatos relacionados

Como a Rule agora é a própria fonte canônica da obrigação no Cursor, não existe uma segunda representação em um adapter. O risco de drift passa a ocorrer quando **outros artefatos repetem, interpretam ou verificam parte dessa obrigação**.

Considere a Rule:

```
.cursor/rules/persistir-antes-de-notificar.mdc
```

```
DEVE persistir o resultado final.
```

```
DEVE confirmar a transação antes da publicação.
```

```
NÃO DEVE notificar conclusão antes da confirmação.
```

Agora imagine que o Playbook ainda contenha uma sequência antiga:

```
Persistir resultado
```

```
↓
```

```
Notificar frontend
```

enquanto a Rule já exige:

```
Persistir resultado
```

```
↓
```

```

Confirmar transação
  ↓
Notificar frontend

```

A Rule continua correta, mas o Playbook passou a coordenar uma sequência incompatível com a obrigação vigente.

O mesmo problema pode ocorrer em Contracts, Skills e Evals:

```

Rule canônica
  ↓
obrigação vigente
  ↓
Playbook / Skill / Contract / Eval
  ↓
interpretação ou verificação compatível

```

Por isso:

- mantenha a obrigação normativa em uma única Rule canônica;
- evite duplicar integralmente seu conteúdo em outros artefatos;
- quando outro artefato depender da Rule, preserve uma referência explícita sempre que isso melhorar a rastreabilidade;
- atualize os artefatos dependentes quando a obrigação mudar;
- utilize Evals para detectar comportamentos que contradigam a Rule vigente.

Por exemplo, o Playbook pode referenciar diretamente:

```
.cursor/rules/persistir-antes-de-notificar.mdc
```

sem precisar copiar toda a justificativa normativa.

O objetivo é preservar uma relação rastreável:

```

Rule canônica
  ↓
artefatos que dependem da obrigação
  ↓
execução
  ↓
Eval verifica o comportamento
  ↓
Evidence registra o observado

```

Assim, a existência de uma única Project Rule elimina o drift entre **Domain Rule e adapter**, mas não elimina o risco de inconsistência entre a obrigação e os artefatos que a utilizam. A governança passa a concentrar-se nessa relação.

2.1.3 Teste mínimo da Rule

Definir uma obrigação em uma Project Rule não demonstra, por si só, que ela será considerada nas tarefas corretas nem que o comportamento final respeitará o que foi declarado. Por isso, o primeiro teste verifica duas coisas: se a Rule **participa quando sua obrigação é aplicável** e se **permanece fora do fluxo quando não é necessária**. Para observar essa diferença, utilizaremos dois cenários opostos:

- **cenário aplicável:** em uma tarefa assíncrona que persista o estado final e depois notifique o cliente, verifique se a solução respeita a ordem **persistir → confirmar → notificar**;
- **cenário não aplicável:** em uma tarefa sem relação com conclusão assíncrona ou notificação, verifique se a Rule não é considerada apenas por estar no mesmo repositório backend.

O teste observa três dimensões diferentes:

aplicabilidade
→ a Rule deveria participar desta tarefa?

resultado funcional
→ a obrigação foi respeitada?

lineage
→ existe evidência suficiente de que a Rule participou?

Essas dimensões não devem ser confundidas. Uma resposta pode respeitar corretamente a obrigação sem que exista evidência suficiente para comprovar que `.cursor/rules/persistir-antes-de-notificar.mdc` participou da execução.

Nesse caso:

Resultado funcional: PASS
Lineage da Rule: INCONCLUSIVE

Da mesma forma, evidência de que a Rule participou não garante, por si só, que a resposta final respeitou a obrigação. O comportamento observado ainda precisa ser comparado com a ordem esperada.

O procedimento reproduzível, com cenário aplicável e não aplicável, expectativa registrada antes da execução, coleta das respostas, evidências de participação e critérios de PASS, FAIL e INCONCLUSIVE, está disponível em [Laboratório — Rule normativa](#).

Com a obrigação explicitada pela Rule, o próximo passo é definir **como verificar objetivamente se ela foi respeitada**. Para isso, as próximas seções partem de um requisito aprovado, relacionam-no a um Contract e suas assertions, definem uma Eval Spec e mostram como os fatos observados durante a execução são preservados em um **Evidence Record**.

A sequência será construída passo a passo:

```

requisito aprovado
    ↓
Rule explicita a obrigação
    ↓
Contract torna condições verificáveis
    ↓
Eval define a expectativa e o verificador
    ↓
execução produz fatos observáveis
    ↓
Evidence preserva o resultado

```

O objetivo é deixar explícitos **a origem da obrigação, o critério utilizado para verificá-la e a evidência que sustenta o resultado**.

2.2 Contracts: o encaixe verificável

A Rule explicita uma obrigação. O **Contract** complementa essa obrigação ao transformar requisitos, restrições e comportamentos esperados em condições que podem ser verificadas na interface ou ao longo do ciclo de execução.

Um Contract é útil quando uma operação precisa produzir um resultado observável e diferentes pessoas, componentes ou agentes precisam concordar sobre o que significa “correto”. Ele não cria autoridade por conta própria: organiza condições derivadas de requisitos, Rules, políticas ou decisões arquiteturais aprovadas.

Considere um cenário comum: gerar um relatório consolidado pode levar alguns minutos. A API aceita a solicitação, cria um job durável, processa o relatório e posteriormente disponibiliza seu resultado. O Contract abaixo não implementa essa operação; ele define o encaixe que

implementação e testes deverão respeitar.

Para manter o ownership estabelecido anteriormente, este Contract permanece junto da Skill que o utiliza:

```
.cursor/  
└─ skills/  
    └─ validate-api-contract/  
        └─ contracts/  
            └─ report-request.contract.yaml
```

BOOK ONTOLOGY — Contract local da Skill:

```
id: CONTRACT-REPORT-001  
name: solicitar-geracao-relatorio  
operation: request-report  
  
input:  
  required: [customerId, period, requestId]  
  
output:  
  accepted:  
    status: 202  
    required: [jobId, statusUrl]  
  
preconditions:  
  - id: customerAccessValidated  
    provenance: external-policy-pending  
  - id: idempotencyPolicyDefined  
    provenance: reference-architecture-decision  
  
postconditions:  
  accepted:  
    - id: durableJobExists  
      provenance: reference-architecture-decision  
    - id: jobStateIsQueryable  
      provenance: approved-requirement  
  
  completed:  
    - id: finalStatePersisted  
      provenance: .cursor/rules/persistir-antes-de-notificar.mdc  
    - id: finalStateIsQueryable  
      provenance: approved-requirement  
  
forbidden:  
  - id: notifyBeforeCommit  
    provenance: .cursor/rules/persistir-antes-de-notificar.mdc  
  - id: finishWithoutEvidence  
    provenance: reference-architecture-governance  
  
observability:  
  correlationFields: [requestId, jobId, correlationId]
```

A proveniência preserva a origem de cada condição. Neste exemplo:

- finalStateIsQueryable deriva de um requisito aprovado;
- finalStatePersisted e notifyBeforeCommit derivam da Rule .cursor/rules/persistir-antes-de-notificar.mdc;
- durableJobExists, idempotencyPolicyDefined e finishWithoutEvidence derivam de decisões ou governança arquitetural;

- `customerAccessValidated` ainda aponta para uma política externa pendente e, portanto, não deve ser apresentado como obrigação definitivamente aprovada até que essa origem seja resolvida.

Essa distinção impede que uma escolha arquitetural, uma hipótese didática ou uma política ainda pendente seja apresentada como obrigação do domínio.

Cada parte responde a uma pergunta concreta:

| Campo | Pergunta respondida | No cenário do relatório |
|---------------------------------------|--|--|
| <code>input</code> | O que o solicitante precisa fornecer? | Cliente, período e identificador idempotente. |
| <code>output.accepted</code> | O que a API devolve ao aceitar a solicitação? | 202, <code>jobId</code> e <code>statusUrl</code> . |
| <code>preconditions</code> | O que precisa ser válido antes de iniciar? | Acesso permitido e política de idempotência conhecida. |
| <code>postconditions.accepted</code> | O que deve ser verdade quando o 202 é devolvido? | Job durável criado e estado do processamento consultável. |
| <code>postconditions.completed</code> | O que deve ser verdade quando o processamento termina? | Estado final persistido e posteriormente consultável. |
| <code>forbidden</code> | O que não pode ocorrer? | Notificar antes da confirmação ou concluir sem evidência suficiente. |
| <code>observability</code> | Como correlacionar a operação? | <code>requestId</code> , <code>jobId</code> e <code>correlationId</code> . |

Separar as pós-condições por estado evita uma ambiguidade importante. Um 202 Accepted informa que a solicitação foi aceita para processamento; ele não significa que o relatório já foi concluído. Por isso, condições que precisam existir **no momento da aceitação** não devem ser misturadas com aquelas que só podem ser verificadas **depois da conclusão**.

O Contract é produzido durante a especificação da operação. Desenvolvedores, arquitetos, analistas ou autores da especificação podem redigi-lo com auxílio de IA, mas cada condição normativa precisa permanecer ligada à fonte responsável pela obrigação — domínio, arquitetura, política ou governança.

O caminho de derivação precisa permanecer visível. Para o requisito de recuperação após desconexão:

```

Requisito aprovado
"0 relatório deve continuar consultável após uma desconexão."
    ↓
Contract
finalStateIsQueryable
    ↓
Implementação esperada
estado persistido + endpoint de consulta
    ↓
Teste
desconectar o cliente + consultar statusUrl
  
```

A mesma rastreabilidade deve existir quando a origem for uma Rule:

```

.cursor/rules/persistir-antes-de-notificar.mdc
    ↓
Obrigação
persistir → confirmar → notificar
    ↓
Contract
finalStatePersisted + notifyBeforeCommit
  
```

```

      ↓
Implementação esperada
commit confirmado antes da notificação
      ↓
Teste
observar e verificar a ordem das operações

```

O Contract, portanto, não substitui a fonte da obrigação. Ele torna **verificável o efeito que essa obrigação deve produzir no cenário aplicável**.

Como aplicar na prática: escolha uma operação observável; identifique suas fontes de autoridade; modele entrada e saída; separe condições de aceitação e conclusão quando houver processamento assíncrono; derive pré-condições, pós-condições e proibições; revise cada condição com o responsável por sua fonte e, somente depois, derive assertions, fixtures e solicitações de implementação.

2.3 Assertions: nomear o que será verificado

O Contract organiza condições verificáveis para uma operação. O próximo passo é decompor essas condições em **assertions**, unidades menores que nomeiam exatamente o que será comparado com o resultado observado.

Uma assertion expressa uma expectativa observável derivada de uma fonte aprovada. Ela não nasce da resposta do Agent nem de um campo retornado pela API; esses elementos fornecem evidências que serão posteriormente comparadas com a expectativa.

O processo de derivação segue esta sequência:

1. **Congele a referência de origem:** registre o artefato, versão ou revisão, seção e responsável pela obrigação.
2. **Separe afirmações atômicas:** cada assertion deve representar uma única obrigação, proibição ou condição verificável.
3. **Selecione o que precisa ser avaliado:** nem toda explicação ou informação de contexto precisa virar uma assertion.
4. **Converta em observável:** substitua expressões vagas, como “com segurança”, por estado, ordem, resposta ou efeito verificável.
5. **Escolha o verificador:** use código para schema, igualdade, ordem e estado; reserve julgamento semântico para significado, equivalência ou contradição que não possam ser determinados de forma confiável por regras determinísticas.
6. **Elimine duplicidades preservando a proveniência:** expectativas equivalentes podem compartilhar uma assertion, desde que todas as fontes relevantes permaneçam registradas.
7. **Atribua um ID estável:** faça isso quando o significado estiver suficientemente consolidado para revisão.
8. **Submeta à aprovação:** o responsável pela fonte da obrigação confirma significado, aplicabilidade e evidência aceitável.

Considere o requisito:

“A solicitação deve continuar consultável após a desconexão, e a notificação não pode antecipar o estado durável.”

A Rule e o Contract derivados anteriormente permitem decompor esse requisito em expectativas verificáveis:

| Fonte aplicável | Classificação | Comportamento observável | Verificador inicial |
|---|--------------------|---|---------------------|
| Notificação não pode antecipar o estado durável. | Obrigaç o de ordem | A confirma  o da persist ncia precede a publica  o da notifica  o. | causalOrderCheck |
| A solicita  o deve continuar consult vel ap s a desconex o. | P s-condi  o | Uma consulta posterior por <code>statusUrl</code> recupera o estado persistido. | statusApiCheck |
| SignalR n o deve ser tratado como fonte dur vel do estado. | Proibi  o | A solu  o preserva uma fonte consult vel de estado independente da notifica  o em tempo real. | approvedLlmJudge |

Os verificadores tamb m possuem responsabilidades diferentes:

`causalOrderCheck`

→ verifica ordem observ vel

`statusApiCheck`

→ verifica estado recuper vel

`approvedLlmJudge`

→ interpreta significado quando a verifica  o determin stica n o   suficiente

Use LLM-as-a-Judge apenas quando a avalia  o depender realmente de interpreta  o sem ntica. Se uma condi  o puder ser verificada por estado, schema, igualdade, sequ ncia ou outro mecanismo determin stico, prefira esse mecanismo.

Com o significado consolidado, o respons vel pela avalia  o atribui IDs est veis e submete o conjunto identificado   aprova  o da fonte respons vel pela obriga  o.

`assertions:`

- `id: ASSERT-REPORT-001`
`sourceOriginal: REQ-REPORT-014`
`sourceCurrent: .cursor/rules/persistir-antes-de-notificar.mdc#obrigacao`
`statement: "A confirma  o da persist ncia deve preceder a notifica  o de conclus o."`
`verifyWith: causalOrderCheck`
- `id: ASSERT-REPORT-002`
`sourceOriginal: REQ-REPORT-014`
`sourceCurrent: CONTRACT-REPORT-001#postconditions.completed`
`statement: "O estado final deve permanecer consult vel ap s a desconex o do cliente."`
`verifyWith: statusApiCheck`
- `id: ASSERT-REPORT-003`
`sourceOriginal: .cursor/rules/persistir-antes-de-notificar.mdc#proibicao`
`sourceCurrent: .cursor/rules/persistir-antes-de-notificar.mdc#proibicao`
`statement: "A solu  o n o deve tratar SignalR como fonte dur vel do estado."`
`verifyWith: approvedLlmJudge`

`sourceOriginal` preserva a origem hist rica da expectativa. `sourceCurrent` aponta para o artefato atualmente utilizado para verificar ou manter aquela obriga  o. Os dois valores podem coincidir quando a pr pria fonte atual originou a assertion.

Essa distin  o preserva lineage quando uma obriga  o evolui:

```

fonte original
  ↓
assertion estável
  ↓
fonte normativa atual
  ↓
verificador
  ↓
evidência observada

```

Os IDs são definidos pelo responsável pela avaliação, com assistência opcional de IA, mas a LLM não cria a autoridade da assertion. A aprovação deve vir do responsável pela fonte da obrigação, que pode ser domínio, arquitetura, segurança ou outra governança aplicável.

Assertions nomeiam **o que esperamos verificar**. Valores retornados pela API, traces, logs, resultados de scripts ou respostas do Agent pertencem ao lado observado da avaliação e serão utilizados posteriormente como evidência.

2.4 Evals: verificar recuperação e aplicação

Depois de transformar requisitos e Rules em Assertions, precisamos verificar se a reorganização do contexto preservou **tanto a estrutura quanto o comportamento esperado durante uma tarefa real**.

Duas perguntas diferentes orientam essa avaliação:

1. **Integridade estrutural:** as obrigações, Contracts, Assertions e relações entre os artefatos continuam preservados e rastreáveis?
2. **Comportamento:** o Agent encontra os recursos aplicáveis e produz uma resposta ou execução compatível com as expectativas aprovadas?

Neste exemplo, o artefato sob avaliação é o Agent `.cursor/agents/backend-development.md`. Ele recebe uma tarefa sobre geração assíncrona de relatório e utiliza o contexto refatorado. O objetivo da Eval é verificar se a nova organização continua funcionando quando colocada em uso, e não apenas se os arquivos existem no repositório.

Uma Eval compara uma **expectativa definida antes da execução** com aquilo que foi efetivamente observado:

```

assertions aprovadas
  ↓
cenário de avaliação
  ↓
execução
  ↓
resultado observado
  ↓
verificadores
  ↓
veredito

```

2.4.1 Camadas de avaliação

Uma suíte pode observar diferentes camadas do comportamento. Elas não representam uma sequência obrigatória para todos os projetos; cada uma responde a um tipo diferente de risco.

| Camada | O que verifica | Exemplo de falha |
|---------------|---|--|
| Lexical | Se termos esperados permitem localizar conteúdo relevante quando existe recuperação lexical. | A consulta não encontra conteúdo relacionado a <code>notify-before-commit</code> . |
| Semântica | Se paráfrases ou formulações equivalentes levam ao conteúdo adequado quando existe recuperação semântica. | “Avisar antes de salvar” não leva à obrigação correspondente. |
| Contextual | Se o artefato adequado participa do cenário correto. | Knowledge consultivo é utilizado como se substituísse uma Rule normativa. |
| Negativa | Se conteúdo não aplicável permanece fora do fluxo quando sua ausência é relevante. | Uma operação síncrona recupera o Playbook assíncrono. |
| Aplicação | Se a orientação recuperada é respeitada no resultado. | A Rule participa, mas a solução notifica antes da confirmação da persistência. |
| Ponta a ponta | Se o comportamento real satisfaz as condições verificáveis. | A API retorna 202, mas nenhum job durável é criado. |

As camadas lexical e semântica só fazem sentido quando o mecanismo avaliado utiliza esse tipo de recuperação. Não introduza um teste de busca lexical ou semântica apenas porque essas categorias existem na suíte.

i O que são critérios semânticos

CrITÉRIOS semânticos exigem interpretar o **significado** de uma resposta, e não apenas comparar texto, campo ou status exato.

Por exemplo, a afirmação de que SignalR é a fonte durável do estado pode aparecer por paráfrase sem repetir essas palavras. Já verificar se um JSON contém `evidenceId`, se um status é 202 ou se um evento ocorreu antes de outro são verificações determinísticas.

Use LLM-as-a-Judge somente quando equivalência de sentido, adequação ou contradição não puderem ser decididas de forma confiável por schema, regra, estado ou assert determinístico.

2.4.2 De onde vêm os valores esperados

Os critérios da Eval não devem ser inventados depois da execução nem derivados somente do conteúdo refatorado.

Antes da refatoração, o curador preserva as obrigações, proibições e critérios relevantes encontrados nas fontes originais. Ambiguidades são resolvidas pelo responsável pela obrigação e transformadas em Assertions aprovadas. Depois da refatoração, cada Assertion é relacionada ao artefato que atualmente preserva aquela expectativa.

```
fontes originais
  ↓
inventário de expectativas
  ↓
resolução de ambiguidades
  ↓
assertions aprovadas
  ↓
refatoração
  ↓
```

vínculo com artefatos atuais
↓
suíte de Evals

Não derive os critérios exclusivamente da versão refatorada. Se uma obrigação desaparecer durante a reorganização, uma Eval construída apenas a partir do resultado novo também poderá ignorar essa perda.

As Assertions definidas na seção anterior podem ser reutilizadas pela suíte:

```
assertions:
- id: ASSERT-REPORT-001
  type: requiredBehavior
  sourceOriginal: REQ-REPORT-014
  sourceCurrent: .cursor/rules/persistir-antes-de-notificar.mdc#obrigacao
  statement: "A confirmação da persistência deve preceder a notificação de conclusão."
  verifyWith: causalOrderCheck

- id: ASSERT-REPORT-002
  type: requiredBehavior
  sourceOriginal: REQ-REPORT-014
  sourceCurrent: CONTRACT-REPORT-001#postconditions.completed
  statement: "O estado final deve permanecer consultável após a desconexão do cliente."
  verifyWith: statusApiCheck

- id: ASSERT-REPORT-003
  type: forbiddenBehavior
  sourceOriginal: .cursor/rules/persistir-antes-de-notificar.mdc#proibicao
  sourceCurrent: .cursor/rules/persistir-antes-de-notificar.mdc#proibicao
  statement: "A solução não deve tratar SignalR como fonte durável do estado."
  verifyWith: approvedLlmJudge
```

Os IDs nomeiam expectativas aprovadas; não são valores retornados pela API nem fatos observados durante a execução.

2.4.3 Eval Spec

Com as Assertions congeladas, podemos definir o cenário que irá exercitá-las.

BOOK ONTOLOGY — Eval Spec:

```
id: EVAL-RECOVERY-ASYNC-001
version: 1.0.0
name: recuperar-aplicar-geracao-assincrona-relatorio

agentUnderEvaluation:
  name: backend-development
  path: .cursor/agents/backend-development.md

corpus:
  id: CORPUS-BACKEND-CONTEXT
  snapshot: 2.1.0

instructions:
  id: PROMPT-BACKEND-ASYNC
  version: 1.4.0

model:
  id: approved-model
  snapshot: 2026-07-15
```

```

query: >
  O relatório pode terminar depois que o cliente se desconectar.
  Proponha o contrato HTTP e o fluxo assíncrono necessários para
  atualizar a tela com segurança e permitir que o resultado
  continue consultável posteriormente.

expected:
  shouldUse:
    - .cursor/playbooks/operacao-assincrona.md
    - .cursor/skills/validate-api-contract/SKILL.md
    - .cursor/skills/validate-api-contract/contracts/report-request.contract.yaml
    - .cursor/rules/persistir-antes-de-notificar.mdc

  mustSatisfy:
    - ASSERT-REPORT-001
    - ASSERT-REPORT-002
    - ASSERT-REPORT-003

evaluationMethod: hybrid

verification:
  artifactParticipation:
    verifier: retrievalTrace

  assertions:
    ASSERT-REPORT-001:
      verifier: causalOrderCheck

    ASSERT-REPORT-002:
      verifier: statusApiCheck

    ASSERT-REPORT-003:
      verifier: approvedLlmJudge
      rubricId: RUBRIC-ASYNC-CLAIMS-001
      rubricVersion: 1.0.0
      modelSnapshot: approved-judge-snapshot-2026-07-15

```

O cenário foi escrito para tornar aplicáveis tanto o fluxo assíncrono quanto a validação de contrato HTTP. Isso evita exigir a recuperação de um artefato que a própria tarefa não justificaria.

Os campos principais possuem responsabilidades diferentes:

| Campo | O que representa | Onde verificar |
|------------------|---|---|
| shouldUse | Artefatos cuja participação é esperada para o cenário. | Traces, registros do host ou outra evidência de recuperação disponível. |
| mustSatisfy | Assertions que precisam permanecer verdadeiras no resultado. | Resposta, código, traces, testes, API ou estado persistido. |
| evaluationMethod | Classe geral dos mecanismos utilizados pela Eval. | Configuração da própria suíte. |
| verification | Verificadores concretos utilizados para produzir os verdicts. | Resultado de cada verificador. |

shouldUse expressa uma expectativa de participação, não uma prova de que o artefato foi utilizado.

Se o host não expuser evidência suficiente para confirmar ou negar essa participação, o resultado correspondente deve ser:

INCONCLUSIVE

e não PASS ou FAIL inferido a partir da qualidade da resposta.

Resultado funcional e lineage continuam independentes:

```
resposta correta
≠
participação comprovada de todos os artefatos

participação comprovada
≠
resultado funcionalmente correto
```

Quando for importante detectar recuperação desnecessária, a Eval também pode declarar artefatos que **não deveriam participar** do cenário. Faça isso somente quando essa ausência for observável e possuir relevância funcional, de custo ou de segurança; conteúdo extra inofensivo não deve reprovar um cenário apenas por estética.

2.4.4 Classe da avaliação e mecanismo de prova

Dois conceitos complementares evitam misturar o **tipo de avaliação** com a **ferramenta utilizada para verificá-la**:

| Conceito | Exemplos | Responsabilidade |
|------------------|--|-----------------------------------|
| evaluationMethod | deterministic, semantic, hybrid | Resume a natureza da avaliação. |
| Verificador | causalOrderCheck, statusApiCheck, jsonSchema, approvedLlmJudge | Executa uma verificação concreta. |

Uma Eval hybrid, por exemplo, pode combinar:

```
causalOrderCheck
→ determinístico

statusApiCheck
→ determinístico

approvedLlmJudge
→ semântico
```



Recuperação não é conformidade

Encontrar o artefato correto não demonstra que a resposta ou a implementação respeitou seu conteúdo.

Por isso, avalie separadamente **participação do contexto**, **satisfação das Assertions** e, quando necessário, **comportamento ponta a ponta**.

2.4.5 Como aplicar na prática

No corpo do capítulo, o fluxo essencial é:

```
congelar expectativas
↓
definir cenário
↓
```

```

executar Agent
  ↓
capturar observado
  ↓
aplicar verificadores
  ↓
classificar resultados
  ↓
registrar Evidence

```

O procedimento reproduzível, com queries diretas, paráfrases, casos negativos, captura de traces, execução dos verificadores e classificação de PASS, FAIL e INCONCLUSIVE, está disponível em [Laboratório — Evals de recuperação e aplicação](#).

★ Dicas de ouro para Evals

- Preserve a relação `assertion` → fonte original → artefato atual.
- Defina expectativas antes da execução.
- Use IA para propor casos, não para criar autoridade normativa.
- Avalie participação e aplicação separadamente.
- Inclua casos negativos quando houver risco real de overfetch.
- Versione corpus, Assertions, instruções, modelo e suíte quando essas dimensões afetarem a reprodutibilidade.
- Use verificadores determinísticos sempre que forem suficientes.
- Não aprobe uma versão com Assertion normativa sem fonte rastreável.

2.4.6 Objetivo SMART do checkpoint

Antes de aprovar a versão 2.1 do corpus, o time de curadoria deve executar 20 casos contra backend-development, comprovar rastreabilidade em 100% das Assertions normativas, obter zero violações das proibições nos casos críticos e produzir um Evidence Record válido para cada execução.

Os números são didáticos. Cada projeto deve calibrar quantidade de casos, limites de falha e profundidade da evidência conforme risco, custo e criticidade.

Com a Eval definida, falta preservar **o que realmente aconteceu em cada execução**. Esse é o papel do Evidence Record, apresentado na próxima seção.

2.5 Evidence Records: tornar a execução auditável

Depois que a Eval define **o que esperamos observar e como verificar**, precisamos preservar o que realmente aconteceu durante a execução. Esse é o papel do **Evidence Record**.

O Evidence Record é o registro estruturado de uma execução avaliada. A Eval define o cenário e os critérios; a execução produz fatos observáveis; os verificadores comparam esses fatos com as Assertions; e o Evidence Record preserva resultados, fontes e vereditos para auditoria, comparação ou diagnóstico.

Ele não é um segundo teste, um log bruto nem uma justificativa narrativa extensa.

O fluxo é:

```

Eval Spec
  ↓
execução
  ↓
fatos observados
  ↓
verificadores

```

↓
vereditos
↓
Evidence Record

As responsabilidades podem ser separadas logicamente:

| Momento | Responsável lógico | Dado produzido |
|--------------------|--------------------|---|
| Antes da execução | Autor da Eval | Cenário, versões, Assertions e expectativas de participação. |
| Durante a execução | Eval Runner | Resposta, artefatos observados, tools, traces, testes e demais fatos disponíveis. |
| Depois da execução | Avaliador | Comparação entre esperado e observado e resultado de cada verificador. |
| Registro final | Evidence Recorder | Evidence Record com resultados, lineage, veredito e observações. |

Essas responsabilidades podem existir no mesmo pipeline ou componente. Os nomes representam **funções lógicas**, não a exigência de quatro agentes ou serviços diferentes. Essa separação ajuda a identificar quem produz cada informação e evita atribuir genericamente toda a avaliação a “um agente”.

2.5.1 Quando usar

Produza um Evidence Record quando precisar:

- comparar versões de contexto, Agent, prompt ou modelo;
- auditar uma execução;
- diagnosticar falhas de recuperação, roteamento ou aplicação;
- avaliar o comportamento de um Agent;
- demonstrar o resultado de um critério de conclusão;
- reconstruir posteriormente por que uma execução recebeu determinado veredito.

Antes de executar a Eval, deixe congelados os elementos que pertencem ao lado **esperado**:

- caso ou pedido avaliado;
- versão da Eval;
- versão ou snapshot do corpus;
- instruções ou prompt utilizados;
- modelo e versão ou snapshot relevantes;
- artefatos cuja participação é esperada, quando isso puder ser observado;
- Assertions que precisam ser satisfeitas;
- verificadores associados a cada Assertion;
- comportamentos proibidos já representados pelas Assertions aplicáveis;
- política de mascaramento ou retenção de dados, quando necessária.

Esses elementos não devem ser reconstruídos a partir da resposta depois da execução.

Durante a execução, capture apenas o que puder ser observado pelo host ou pelos mecanismos de avaliação, por exemplo:

resposta produzida
artefatos recuperados ou utilizados
tool calls
resultados de scripts
traces
status e respostas de APIs

estado persistido
resultados de testes

A ausência de observabilidade também é um resultado relevante. Se a Eval espera comprovar a participação de uma Rule ou Skill, mas o host não fornece informação suficiente para confirmar ou negar essa transição, o Evidence Record deve preservar essa limitação como **INCONCLUSIVE**, em vez de inferir participação a partir da qualidade da resposta.

A separação essencial permanece:

esperado
→ definido pela Eval e pelas Assertions

observado
→ produzido durante a execução

veredito
→ resultado da comparação

Evidence Record
→ preserva os três com rastreabilidade

Essa distinção permite reconstruir posteriormente não apenas **se a execução passou ou falhou**, mas também **qual expectativa foi avaliada, qual evidência sustentou a decisão e onde a observabilidade foi insuficiente**.

2.5.2 Exercício copiável: auditar uma execução

O prompt abaixo recebe os dados coletados pelo Eval Runner e atua como **avaliador semântico e registrador da evidência**. Ele não executa novamente o Agent sob avaliação, não substitui verificadores determinísticos e não solicita raciocínio interno da LLM.

Seu papel é interpretar apenas os critérios que realmente dependem de significado e consolidar, junto dos resultados determinísticos já disponíveis, um Evidence Record rastreável.

Auditor de Contexto

Objetivo

Avaliar a execução fornecida comparando expectativas previamente definidas com evidências observadas e produzir um Evidence Record conforme o schema informado.

Entradas

- CASO: {{pedido_do_usuario}}
- ARTEFATOS_ESPERADOS: {{ids_ou_caminhos}}
- ARTEFATOS_OBSERVADOS: {{ids_caminhos_trechos_ou_traces}}
- ASSERTIONS_ESPERADAS: {{ids_fontes_enunciados_e_verificadores}}
- RESPOSTA_DO_AGENTE: {{texto_plano_ou_resultado}}
- TOOLS_EXECUTADAS: {{nome_entradas_saidas_status}}
- TESTES_EXECUTADOS: {{ids_e_resultados}}
- RESULTADOS_DETERMINISTICOS: {{verificador_assertion_resultado_evidencia}}
- RESULTADO_OBSERVADO: {{estado_ou_saida_final}}
- CRITERIOS_SEMANTICOS: {{assertions_que_exigem_interpretacao}}
- SCHEMA_EVIDENCE_RECORD: {{schema_json}}

Regras de avaliação

1. Use somente as entradas fornecidas.
2. Não invente arquivo, assertion, execução, teste, resultado ou transição ausente.

3. Preserve os resultados dos verificadores determinísticos; não os substitua por julgamento semântico.
4. Use julgamento semântico somente para os critérios listados em ``CRITERIOS_SEMANTICOS``.
5. Avalie resultado funcional e lineage separadamente.
6. Não trate uma resposta correta como prova de que determinado artefato participou da execução.
7. Marque ``PASS`` quando a evidência disponível sustentar o critério avaliado.
8. Marque ``FAIL`` quando a evidência contradisser o critério ou demonstrar suficientemente sua violação.
9. Marque ``INCONCLUSIVE`` quando a evidência disponível não permitir confirmar nem negar o critério.
10. Diferencie falhas de recuperação, roteamento, aplicação, tool, Contract, verificador e resultado funcional.
11. Não atribua a falha a um artefato específico sem evidência suficiente para sustentar essa relação.
12. Não exponha raciocínio interno; registre apenas critérios, evidências observáveis, resultados e limitações.

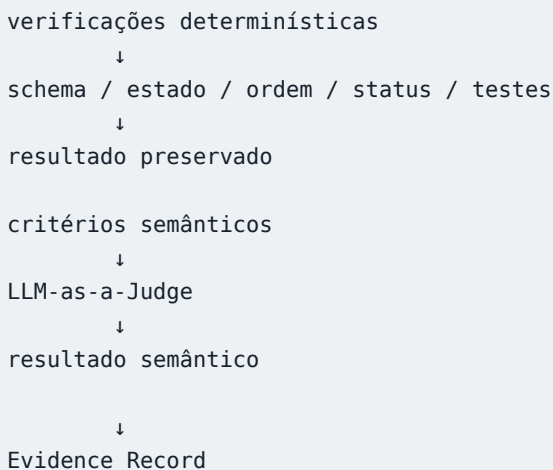
Saída obrigatória

Produza somente o Evidence Record JSON conforme ``SCHEMA_EVIDENCE_RECORD``.

O registro deve preservar, quando previstos pelo schema:

- resultado funcional;
- resultado de lineage;
- resultado de cada assertion;
- evidências utilizadas;
- evidências ausentes;
- limitações de observabilidade;
- próxima ação de investigação ou correção, somente quando sustentada pelos dados disponíveis.

A separação entre verificadores deve permanecer explícita:



Para executar o exercício:

1. congele a Eval Spec, as Assertions e as versões relevantes antes da execução;
2. execute o caso contra `.cursor/agents/backend-development.md` usando o corpus definido pela Eval;
3. capture os artefatos observados, resposta, tools, traces, testes e estado resultante;
4. execute primeiro os verificadores determinísticos associados às Assertions;
5. preencha cada `{{placeholder}}` somente com dados coletados ou resultados produzidos pelos verificadores;
6. forneça ao avaliador semântico apenas os critérios que realmente exigem interpretação;

7. gere o Evidence Record conforme o schema definido;
8. valide por código a estrutura do JSON produzido;
9. preserve o Evidence Record junto das versões necessárias para reconstruir a execução.

Se determinada transição não puder ser observada, registre essa limitação. Por exemplo:

Resultado funcional: PASS

Lineage da Rule: INCONCLUSIVE

Esse resultado é válido: a ausência de evidência suficiente para comprovar lineage não transforma automaticamente uma execução funcionalmente correta em FAIL.

★ **Kit copiável no último capítulo**

O Laboratório 3 deste workbook concentra os templates completos utilizados neste fluxo, incluindo Agent, Skill, Assertions, Eval Spec e Evidence Record. Esta seção ensina como os artefatos se conectam; o kit final reúne as versões extensas para reutilização.

! **LLM-as-a-Judge não substitui verificadores determinísticos**

O julgamento por LLM é útil quando o critério depende de equivalência de significado, adequação ou contradição semântica, mas pode variar entre modelos e execuções. Sempre que schema, estado, igualdade, ordem, status ou outro critério puder ser verificado de forma determinística, prefira código. Use LLM-as-a-Judge apenas para a parcela que realmente exigir interpretação e mantenha revisão humana amostral quando o risco justificar.

2.5.3 Exemplo preenchido do exercício

O bloco abaixo exemplifica os dados fornecidos ao avaliador depois da execução.

- artefatosEsperados, assertionsEsperadas e criteriosSemanticos vêm da Eval Spec;
- artefatosObservados, respostaDoAgente, toolsExecutadas, testesExecutados e resultadoObservado vêm da execução;
- resultadosDeterministicos são produzidos pelos verificadores executados antes do julgamento semântico.

caso: >

Propor o contrato HTTP e o fluxo assíncrono para geração de um relatório que possa terminar depois que o cliente se desconectar e permanecer consultável posteriormente.

artefatosEsperados:

- .cursor/playbooks/operacao-assincrona.md
- .cursor/skills/validate-api-contract/SKILL.md
- .cursor/skills/validate-api-contract/contracts/report-request.contract.yaml
- .cursor/rules/persistir-antes-de-notificar.mdc

artefatosObservados:

- path: .cursor/playbooks/operacao-assincrona.md
evidence: retrieval-trace
trecho: "Persistir o resultado, confirmar a transação e depois notificar o frontend."
- path: .cursor/skills/validate-api-contract/SKILL.md
evidence: retrieval-trace
trecho: "Compare entrada, saída, erros, headers e pós-condições aplicáveis."
- path: .cursor/skills/validate-api-contract/contracts/report-request.contract.yaml

```
evidence: retrieval-trace
trecho: "finalStateIsQueryable"
```

- path: .cursor/rules/persistir-antes-de-notificar.mdc
evidence: retrieval-trace
trecho: "Persista e confirme o estado final antes de publicar a notificação de conclusão."

assertionsEsperadas:

- id: ASSERT-REPORT-001
source: .cursor/rules/persistir-antes-de-notificar.mdc#obrigacao
statement: "A confirmação da persistência deve preceder a notificação de conclusão."
verifyWith: causalOrderCheck
- id: ASSERT-REPORT-002
source: CONTRACT-REPORT-001#postconditions.completed
statement: "O estado final deve permanecer consultável após a desconexão do cliente."
verifyWith: statusApiCheck
- id: ASSERT-REPORT-003
source: .cursor/rules/persistir-antes-de-notificar.mdc#proibicao
statement: "A solução não deve tratar SignalR como fonte durável do estado."
verifyWith: approvedLlmJudge

respostaDoAgente: >

Persista o relatório e confirme a transação antes de publicar a notificação SignalR. A notificação atualiza a interface, mas o estado durável permanece consultável por statusUrl caso o cliente se desconecte ou não receba o evento.

toolsExecutadas:

- name: complete_report
status: success
correlationId: corr-9812

testesExecutados:

- id: TEST-E2E-REPORT-007
result: passed

resultadosDeterministicos:

- assertionId: ASSERT-REPORT-001
verifier: causalOrderCheck
result: PASS
evidence:
 - result_persisted
 - transaction_committed
 - notification_published
- assertionId: ASSERT-REPORT-002
verifier: statusApiCheck
result: PASS
evidence:
 - statusUrl: /reports/job-742/status
 - returnedState: completed

critériosSemanticos:

- assertionId: ASSERT-REPORT-003
verifier: approvedLlmJudge
statement: "A solução não deve tratar SignalR como fonte durável do estado."

```

resultadoObservado:
  jobId: job-742
  durableStatus: completed
  eventOrder:
    - result_persisted
    - transaction_committed
    - notification_published
  signalrEvent: delivered
  statusApi: consistent

```

O schemaEvidenceRecord também deve ser fornecido ao avaliador conforme definido no exercício anterior. Ele não é repetido neste exemplo para evitar duplicar um template extenso que será apresentado no kit final.

Observe que os três tipos de informação permanecem separados:

```

esperado
→ artefatos + Assertions + verificadores

observado
→ resposta + tools + testes + estado + traces

resultado dos verificadores
→ PASS / FAIL / INCONCLUSIVE

```

O avaliador semântico não deve reinterpretar causalOrderCheck ou statusApiCheck. Neste exemplo, sua responsabilidade é avaliar ASSERT-REPORT-003 e consolidar os resultados disponíveis no Evidence Record.

2.5.4 Exemplo de saída

O Evidence Record abaixo consolida os fatos coletados e os resultados dos verificadores do exemplo anterior. Ele preserva separadamente **o que era esperado, o que foi observado e qual veredito foi produzido em cada dimensão**.

```

{
  "schemaVersion": "1.0.0",
  "evidenceId": "EV-2026-0042",
  "requestId": "REQ-2026-0188",
  "correlationId": "corr-9812",
  "eval": {
    "id": "EVAL-RECOVERY-ASYNC-001",
    "version": "1.0.0"
  },
  "context": {
    "agent": "backend-development",
    "corpusSnapshot": "2.1.0",
    "instructionsVersion": "1.4.0",
    "modelSnapshot": "2026-07-15"
  },
  "expected": {
    "artifacts": [
      ".cursor/playbooks/operacao-assincrona.md",
      ".cursor/skills/validate-api-contract/SKILL.md",
      ".cursor/skills/validate-api-contract/contracts/report-request.contract.yaml",
      ".cursor/rules/persistir-antes-de-notificar.mdc"
    ],
    "assertions": [

```

```
"ASSERT-REPORT-001",
"ASSERT-REPORT-002",
"ASSERT-REPORT-003"
]
},
"observed": {
  "artifacts": [
    {
      "path": ".cursor/playbooks/operacao-assincrona.md",
      "evidence": "retrieval-trace",
      "result": "PASS"
    },
    {
      "path": ".cursor/skills/validate-api-contract/SKILL.md",
      "evidence": "retrieval-trace",
      "result": "PASS"
    },
    {
      "path": ".cursor/skills/validate-api-contract/contracts/report-request.contract.yaml",
      "evidence": "retrieval-trace",
      "result": "PASS"
    },
    {
      "path": ".cursor/rules/persistir-antes-de-notificar.mdc",
      "evidence": "retrieval-trace",
      "result": "PASS"
    }
  ],
  "assertions": [
    {
      "id": "ASSERT-REPORT-001",
      "verifier": "causalOrderCheck",
      "result": "PASS"
    },
    {
      "id": "ASSERT-REPORT-002",
      "verifier": "statusApiCheck",
      "result": "PASS"
    },
    {
      "id": "ASSERT-REPORT-003",
      "verifier": "approvedLlmJudge",
      "result": "PASS"
    }
  ],
  "runtime": {
    "durableStatus": "completed",
    "eventOrder": [
      "result_persisted",
      "transaction_committed",
      "notification_published"
    ],
    "signalrEvent": "delivered",
    "statusApi": "consistent"
  }
},
"verdict": {
```

```

    "functional": "PASS",
    "lineage": "PASS",
    "overall": "PASS"
  },
  "tests": [
    {
      "id": "TEST-E2E-REPORT-007",
      "result": "PASS"
    }
  ],
  "residualRisks": [],
  "timestamp": "2026-08-02T14:30:00Z"
}

```

Um registro útil deve permitir responder, sem reconstrução manual demorada:

- qual execução foi avaliada;
- quais versões de Agent, corpus, instruções, modelo e Eval participaram;
- quais artefatos eram esperados;
- quais artefatos possuem evidência observável de participação;
- quais Assertions foram verificadas;
- quais verificadores produziram cada resultado;
- qual estado foi observado ao final;
- quais riscos ou limitações permaneceram.

O campo verdict preserva dimensões diferentes:

```

functional
→ o comportamento observado satisfaz as Assertions?

lineage
→ existe evidência suficiente sobre a participação esperada dos artefatos?

overall
→ qual é o resultado final segundo a política definida pela Eval?

```

O resultado overall não deve ocultar os demais. Uma política de aceitação pode exigir lineage comprovado em determinado cenário, enquanto outra pode aceitar resultado funcional com lineage INCONCLUSIVE. Essa decisão precisa existir na Eval Spec antes da execução.

Considere, por exemplo, que o Playbook era esperado, mas sua participação não pôde ser comprovada.

Se o host não fornecer observabilidade suficiente:

```

Resultado funcional: PASS
Lineage do Playbook: INCONCLUSIVE

```

Se, ao contrário, houver evidência suficiente de que o Playbook **não participou**, embora sua participação fosse exigida pela Eval, o resultado de lineage correspondente pode ser FAIL.

Portanto:

```

ausência de evidência
≠
evidência de ausência

```

A qualidade da resposta final também não deve ser usada para preencher retrospectivamente `observed.artifacts`.

2.5.5 Como interpretar os resultados

| Resultado | Significado | Próxima ação |
|--------------|---|---|
| PASS | A evidência disponível sustenta o critério avaliado. | Preservar o registro e utilizá-lo como baseline quando aplicável. |
| FAIL | A evidência contradiz o critério ou demonstra suficientemente sua violação. | Localizar a camada sustentada pela evidência, corrigir a origem e executar uma nova rodada. |
| INCONCLUSIVE | A evidência disponível não permite confirmar nem negar o critério. | Melhorar observabilidade ou coletar o dado ausente antes de atribuir causa. |

Esses resultados devem ser interpretados **por critério e por dimensão**. Um FAIL em uma Assertion não transforma automaticamente todas as transições de lineage em FAIL, assim como lineage INCONCLUSIVE não invalida automaticamente um resultado funcional comprovado.

2.5.6 Diagnóstico por camada

O Evidence Record ajuda a indicar **onde investigar primeiro**, mas o sintoma observado não prova sozinho sua causa. O diagnóstico deve começar pelo critério afetado e avançar somente até onde a evidência permitir.

| Sintoma observado | Camada a investigar | Próxima verificação |
|--|--|---|
| Artefato esperado não possui evidência de participação | Descoberta, recuperação ou observabilidade | Verificar primeiro se existe evidência suficiente para distinguir FAIL de INCONCLUSIVE; depois revisar sinais de aplicabilidade e mecanismo de recuperação. |
| Artefato participou, mas a obrigação foi violada | Aplicação | Comparar a Assertion, o conteúdo recuperado e o resultado produzido antes de alterar instruções. |
| Tool retornou erro ou dado incompleto | Execução | Verificar comportamento esperado para erro, contrato da tool, retry, timeout e fallback previsto. |
| Teste passou, mas o estado final divergiu | Teste, verificador ou ponta a ponta | Revisar fixture, Assertion, verificador e fonte observada do estado. |
| Avaliador não consegue decidir | Evidência ou observabilidade | Identificar qual dado está ausente e instrumentar sua coleta quando necessário. |

Uma falha de tool, por exemplo, não significa automaticamente FAIL da tarefa. Se a Eval espera que o Agent reconheça o erro e execute um fallback válido, o comportamento pode continuar conforme o esperado.

O veredito deve resultar da comparação com a Assertion correspondente, e não simplesmente da existência de um erro intermediário.

Da mesma forma, não altere imediatamente prompt, Rule, Skill ou Playbook apenas porque um deles parece ser a causa mais provável. Preserve a cadeia diagnóstica:

```

critério afetado
  ↓
evidência observada
  ↓
camada a investigar

```

```

      ↓
artefato responsável, se comprovável
      ↓
correção
      ↓
nova execução

```

A expressão **camada a investigar** é intencional. Enquanto a evidência não demonstrar a origem do problema, o registro deve apontar uma direção de investigação, não atribuir uma causa definitiva.

2.5.7 Conteúdo mínimo e retenção

Um Evidence Record também não deve armazenar indiscriminadamente prompts internos, dados pessoais, payloads completos ou informações que não sejam necessárias para reconstruir a avaliação.

Preserve somente o necessário, conforme as políticas aplicáveis de segurança, retenção e privacidade:

- identificadores e `correlationId`;
- versões e snapshots relevantes;
- estados observados;
- resultados de testes e verificadores;
- referências ou hashes quando suficientes;
- trechos mínimos necessários para sustentar determinado resultado;
- limitações de observabilidade que afetaram a avaliação.

Evite registrar cadeia de pensamento ou outros dados internos desnecessários para reproduzir o veredito.

2.5.8 Checklist de qualidade do Evidence Record

Antes de considerar o registro concluído, verifique:

- `expected`, `observed` e `verdict` permanecem separados;
- resultado funcional e lineage permanecem em dimensões distintas;
- fatos são capturados automaticamente quando o harness permitir;
- artefatos, tools e testes só aparecem como observados quando existe evidência correspondente;
- lacunas de instrumentação resultam em `INCONCLUSIVE` quando impedirem uma decisão;
- `Eval`, `corpus`, `instructions`, modelo e `correlationId` estão identificados quando necessários para reprodução;
- cada Assertion preserva o verificador e seu resultado;
- causa ou responsabilidade não é atribuída sem evidência suficiente;
- dados sensíveis desnecessários não foram preservados;
- outra pessoa consegue reconstruir o veredito usando apenas o registro e as fontes referenciadas.

★ Teste final de auditabilidade

Se outra pessoa não consegue reconstruir **o que era esperado, o que foi observado e por que o veredito foi produzido**, o Evidence Record ainda está incompleto.

2.6 Reduzindo omissão e alucinação operacional

Esta seção não introduz um novo artefato. Ela reúne propriedades que devem aparecer nos elementos anteriores antes de considerar o contexto suficientemente preparado para uso.

Estrutura, por si só, não elimina alucinação ou omissão. O objetivo é reduzir a margem para decisões implícitas, tornar restrições explícitas e criar pontos que possam ser verificados depois da execução.

Use quatro elementos complementares:

1. **Obrigação:** o que precisa acontecer.
2. **Proibição:** o que não pode acontecer.
3. **Fallback:** o que fazer quando o caminho principal falha.
4. **Evidência:** como demonstrar que o comportamento esperado ocorreu.

O checklist abaixo é uma **heurística de revisão**, não um novo arquivo obrigatório da ontologia:

```
# Checklist de conclusão
```

```
## Verificações
```

- [] O Contract aplicável foi identificado.
- [] As pré-condições foram verificadas.
- [] Nenhuma proibição aplicável foi violada.
- [] O fallback necessário foi preservado.
- [] A conclusão está sustentada por resultados observáveis.

Checklist é barreira, não prova: ele ajuda a evitar omissões previsíveis, mas marcar uma caixa não comprova que o comportamento ocorreu. A prova precisa vir de testes, estados, traces, respostas, resultados de verificadores ou outras evidências observáveis.

2.7 Fechando o capítulo: do pedido à evidência

O documento backend caótico apresentado no início agora possui responsabilidades mais explícitas. A tabela resume o sistema construído; ela não introduz uma nova taxonomia.

| Artefato | Pergunta principal | Evidência mínima |
|-----------------|--|--|
| Agent | Quem decide o próximo passo e quando parar? | Roteamento, delegação ou condição de parada observável quando disponível. |
| Playbook | Qual sequência coordena o objetivo? | Etapas aplicáveis e transições executadas ou representadas no resultado. |
| Skill | Qual capacidade especializada precisa ser executada? | Entrada, execução aplicável, saída e resultado observável. |
| Knowledge | Qual informação apoia a interpretação ou decisão? | Conteúdo recuperado e evidência suficiente de sua utilização, quando observável. |
| Rule | Qual obrigação ou proibição precisa ser respeitada? | Comportamento observado comparável à obrigação aplicável. |
| Contract | Quais condições tornam a operação verificável? | Entradas, saídas, pré-condições, pós-condições e proibições comparadas ao observado. |
| Eval | Como o esperado será comparado com o observado? | Assertions, verificadores e resultados por critério. |
| Evidence Record | O que aconteceu e como o veredito pode ser reconstruído? | Expected, observed, resultados dos verificadores, lineage e verdict preservados. |

A cadeia completa fica:

```
pedido
↓
Agent decide e roteia
↓
Playbook coordena quando aplicável
↓
Skill executa capacidades especializadas
↓
Knowledge apoia interpretação quando necessário
↓
Rule restringe o comportamento
↓
Contract fornece condições verificáveis
↓
execução produz fatos observáveis
↓
Eval compara esperado e observado
↓
Evidence Record preserva o resultado
```

Nem todos os artefatos precisam participar de todas as tarefas. O fluxo representa responsabilidades possíveis; progressive disclosure e sinais de aplicabilidade determinam quais recursos realmente precisam entrar em cada execução.

O capítulo começou perguntando **quem consome cada artefato e por qual mecanismo ele participa da execução**. A resposta agora não depende apenas da presença de arquivos: cada responsabilidade precisa possuir um consumidor, uma forma de participação e, quando necessário, evidência suficiente para demonstrar seu efeito.

No próximo capítulo, essa cadeia deixa de aparecer em exemplos isolados e passa a operar sobre um único caso semi-completo. Se algum elo necessário ainda não puder ser explicado, executado ou verificado, registre a lacuna antes de avançar.

Capítulo 3

Laboratório 2 — estudo de caso: operação assíncrona

A Parte III apresentou Agent, Playbook, Skill, Knowledge, Rule, Contract, Eval e Evidence Record separadamente para deixar claras suas responsabilidades. Agora vamos observar **como essas peças cooperam quando uma única tarefa exige várias delas ao mesmo tempo**.

A geração assíncrona de relatório será usada como cenário de integração. O objetivo não é ensinar arquitetura assíncrona em profundidade, mas acompanhar como os artefatos de Context Engineering participam de um mesmo fluxo, desde o pedido inicial até a avaliação e o registro da evidência.

Este capítulo reúne, portanto, os conceitos anteriores em um exemplo **semi-completo**. Ele cobre apenas as partes necessárias para demonstrar essa cooperação sem transformar o estudo de caso em uma arquitetura integral de produção.

| Incluído no exemplo | Omitido deliberadamente |
|---------------------------------------|---|
| Contract HTTP e idempotência básica | Autenticação e autorização detalhadas |
| Job, estados e persistência | Dimensionamento e topologia de infraestrutura |
| SignalR e reconciliação por API | Deploy, disaster recovery e retenção |
| Edge cases, testes e Evidence Records | Políticas corporativas completas de segurança e privacidade |

O recorte acompanha uma única tarefa do início ao fim:

```
requisito
→ aceitação da solicitação
→ criação e processamento do job
→ persistência da conclusão
→ notificação e reconciliação
→ avaliação
→ Evidence Record
```

Ao longo desse caminho, o foco será observar **qual artefato participa de cada decisão, por que ele se torna aplicável e qual evidência permite verificar seu efeito**.



Por que o exemplo é semi-completo

As camadas omitidas continuam importantes. Elas foram retiradas para que o leitor consiga observar com clareza a integração entre os artefatos de Context Engineering sem transformar o capítulo em uma especificação completa de backend distribuído.

3.1 O requisito

Uma API inicia a geração de um relatório consolidado. O processamento pode durar minutos. O front-end precisa acompanhar o processamento e atualizar a tela quando o relatório ficar pronto, mesmo que a conexão em tempo real seja temporariamente perdida.

O desafio pode ser resumido em três necessidades:

```

aceitar o trabalho
    ↓
processar de forma durável
    ↓
permitir atualização e recuperação do estado
  
```

Esse único requisito mobiliza responsabilidades diferentes dos artefatos apresentados anteriormente:

| Responsabilidade | Artefato ou mecanismo principal |
|---|-----------------------------------|
| Definir aceitação, estado consultável e comportamento observável da API | Contract |
| Coordenar a sequência da operação assíncrona | Playbook |
| Executar validações especializadas do contrato | Skill |
| Preservar obrigações como persistir antes de notificar | Rule |
| Interpretar o papel e os limites do SignalR | Knowledge |
| Atualizar o cliente conectado | Canal SignalR |
| Permitir recuperação após desconexão | Estado durável + consulta por API |
| Verificar expectativas e preservar o resultado | Eval + Evidence Record |

i Uma tarefa, responsabilidades diferentes

A tabela não afirma que todos esses elementos utilizam o mesmo mecanismo nem que precisam ser carregados ao mesmo tempo.

Ela mostra como **uma única tarefa atravessa responsabilidades diferentes**, cada uma assumida pelo artefato ou mecanismo mais adequado.

3.2 Contrato de aceitação

O Contract apresentado anteriormente separou condições de **aceitação** e **conclusão**. Neste estudo de caso, vamos observar esses dois momentos separadamente para entender o que precisa ser verificável em cada etapa da mesma operação.

O primeiro recorte governa o início do fluxo: o cliente envia a solicitação e a API confirma que o trabalho foi aceito para processamento. 202 Accepted não significa que o relatório ficou pronto.

Neste estudo, a aceitação só é considerada válida quando já existe um job durável e uma referência que permita acompanhar seu estado posteriormente.

Esse recorte protege três decisões:

- repetir a mesma solicitação não deve criar trabalho lógico duplicado;
- a API não deve responder 202 antes de existir um job recuperável;
- o cliente deve receber uma referência que permita consultar o processamento depois da resposta inicial.

```

POST /reports
Idempotency-Key: 2ca9...
  
```

```

HTTP/1.1 202 Accepted
Location: /reports/jobs/01J...
X-Correlation-Id: corr-9812

{
  "jobId": "01J...",
  "status": "queued",
  "statusUrl": "/reports/jobs/01J..."
}

```

3.2.1 Condições verificáveis da aceitação

- a entrada é validada antes da criação do job;
- repetir a solicitação com o mesmo identificador idempotente não cria trabalho lógico duplicado;
- a resposta 202 contém jobId e statusUrl;
- o job existe em armazenamento durável antes da resposta;
- correlationId permite relacionar a aceitação às etapas posteriores da mesma execução.

Essas condições podem originar Assertions específicas. O Contract define **o que deve ser verdade**; a Eval determinará posteriormente **como cada condição será verificada**.

3.3 Contrato de progresso e conclusão

O segundo recorte acompanha o que acontece depois da aceitação. Ele define as condições necessárias para que o processamento evolua até um estado final e para que o front-end consiga combinar atualização em tempo real com reconciliação pelo estado persistido.

O evento SignalR reduz a latência percebida, mas não comprova sozinho que a operação terminou. Neste estudo, a Rule `.cursor/rules/persistir-antes-de-notificar.mdc` preserva a ordem:

```

persistir estado final
    ↓
confirmar transação
    ↓
publicar notificação

```

Depois disso, o cliente pode utilizar a notificação para atualizar rapidamente a interface ou consultar statusUrl para reconciliar o estado.

Um evento de conclusão pode assumir esta forma:

```

{
  "eventType": "report.completed",
  "jobId": "01J...",
  "version": 7,
  "occurredAt": "2026-08-02T14:30:00Z",
  "status": "completed",
  "resultUrl": "/reports/jobs/01J.../result"
}

```

O campo version representa, neste exemplo, uma versão crescente do estado do job. Ele permite ao consumidor reconhecer eventos atrasados e evitar que uma atualização antiga sobrescreva um estado mais recente.

3.3.1 Condições verificáveis da conclusão

Para considerar a conclusão confiável, o Contract precisa tornar observáveis as seguintes propriedades:

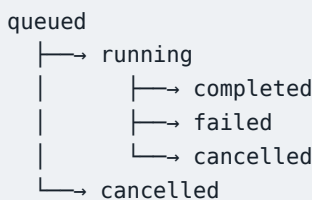
- o estado `completed` é persistido antes da publicação do evento;
- a confirmação da persistência ocorre antes da notificação de conclusão;
- o evento possui versão crescente ou outro mecanismo capaz de identificar atualizações obsoletas;
- eventos duplicados não produzem efeitos lógicos duplicados no front-end;
- uma atualização atrasada não faz a interface regredir para um estado anterior;
- falha na entrega da notificação não reverte o resultado já persistido;
- a consulta por `statusUrl` retorna o estado durável correspondente ao job;
- `jobId` e `correlationId` permitem relacionar processamento, persistência, notificação e consulta.

Essas condições ainda não constituem Evidence. Elas definem **o que esperamos observar** e podem ser decompostas em Assertions verificadas por mecanismos determinísticos ou semânticos, conforme a natureza de cada critério.

3.4 Estados e transições

O Contract também precisa deixar explícitos os estados relevantes e as transições permitidas. Sem essa definição, termos como `queued`, `running`, `completed` ou `failed` podem ser interpretados de maneiras diferentes por API, worker, front-end e testes.

Neste estudo, o fluxo simplificado é:



`completed`, `failed` e `cancelled` são estados finais neste modelo.

Algumas invariantes protegem essas transições:

- uma operação em estado final não retorna silenciosamente para `running`;
- cancelamento concorrente possui resultado definido;
- retry da mesma execução não cria um segundo trabalho logicamente distinto;
- eventos duplicados não duplicam efeitos no consumidor;
- eventos com versão anterior ao estado já conhecido não fazem a interface regredir;
- timeout ou falha transitória de infraestrutura não deve ser automaticamente classificado como falha do domínio.

Essas invariantes conectam o modelo de estados aos artefatos já apresentados:

```

Contract
→ define estados e condições observáveis

Rule
→ preserva obrigações e proibições

Playbook
→ coordena a sequência operacional

Assertions
→ nomeiam propriedades que serão verificadas
  
```

Eval

→ aplica os verificadores

Evidence Record

→ preserva os resultados observados

O objetivo não é transformar o Contract em implementação. Ele deve fornecer informação suficiente para que API, worker, front-end e avaliação compartilhem **o mesmo significado para aceitação, progresso, conclusão e recuperação**.

3.5 Edge cases obrigatórios neste recorte

O fluxo principal mostra o caminho esperado, mas uma operação assíncrona confiável também precisa definir o comportamento nos desvios previsíveis. Neste estudo, os edge cases abaixo exercitam principalmente idempotência, durabilidade, ordenação, reconciliação e concorrência:

| Caso | Resultado esperado |
|---|---|
| Cliente repete o POST com a mesma chave de idempotência | A mesma operação lógica é preservada; nenhum trabalho duplicado é criado. |
| Worker cai durante o processamento | O job volta a ser elegível para processamento ou termina em falha controlada, conforme a política definida. |
| Commit conclui e SignalR falha | O estado final permanece consultável; a falha da notificação não reverte a conclusão persistida. |
| Evento chega duas vezes | O front-end mantém uma única representação lógica do estado. |
| Evento antigo chega depois de um mais recente | A versão mais nova prevalece e a interface não regride. |
| Usuário reconecta | A tela consulta <code>statusUrl</code> e reconcilia o estado atual. |
| Cancelamento disputa com conclusão | A política de transição determina um único estado final válido. |

Autenticação e autorização detalhadas permanecem fora do recorte deste estudo. Em uma arquitetura de produção, consultas a jobs de outros usuários, vazamento de existência de recursos e políticas de acesso também precisam de cenários específicos.

Esses casos não são apenas exemplos de teste. Cada um deve poder ser relacionado a uma condição do Contract, Rule, Assertion ou política aplicável antes de participar de uma Eval.

3.6 Massa progressiva de testes

Em vez de começar diretamente por cenários ponta a ponta, organize a massa em camadas que aumentam progressivamente o número de componentes envolvidos:

- **Camada 1 — Contract:** payload mínimo válido, campos obrigatórios ausentes, formato inválido e chave de idempotência repetida;
- **Camada 2 — máquina de estados:** transições válidas e proibidas, retry após falha transitória, cancelamento concorrente e versão fora de ordem;
- **Camada 3 — integração:** fila, persistência ou hub indisponível, evento duplicado e reinício do worker;
- **Camada 4 — ponta a ponta:** cliente conectado recebe conclusão, cliente desconectado recupera o estado pela API, múltiplas instâncias da interface convergem e a telemetria correlaciona solicitação, job, commit e publicação.

A progressão ajuda a localizar falhas. Se uma condição já falha no Contract ou na máquina de estados, um teste ponta a ponta pode confirmar o problema, mas tende a fornecer uma origem

menos precisa.

3.7 Captura operacional observada

Depois da execução, o Eval Runner precisa preservar os fatos que serão comparados com as Assertions. O YAML abaixo representa uma **captura operacional bruta** desse momento.

Ele ainda não é um Evidence Record: contém o observado, mas não realiza a comparação com expectativas versionadas nem produz um veredito.

```
operation: report-generation-async
jobId: 01J...
correlationId: corr-9812
contract: CONTRACT-REPORT-001

stateTransitions:
  - queued
  - running
  - completed

durableState:
  status: completed
  version: 7
  persistedSequence: 451

notification:
  channel: signalr
  outcome: delivered
  publishedSequence: 452

fallbackCheck:
  statusUrl: /reports/jobs/01J.../status
  statusApi: consistent

tests:
  - id: TEST-E2E-REPORT-007
    result: PASS
```

A captura permite observar, por exemplo:

```
persistedSequence: 451
  ↓
publishedSequence: 452
```

Esses valores podem ser utilizados por `causalOrderCheck` para verificar a Assertion que exige persistência e confirmação antes da notificação.

O registro não precisa conter raciocínio interno do modelo. Ele preserva somente fatos observáveis necessários para avaliação e auditoria.

3.8 Fechando o ciclo do caso

Até aqui, o estudo acompanhou duas linhas que agora precisam se encontrar:

EXPECTATIVA

```
requisito
→ Rule / Contract
→ Assertions
```

→ Eval Spec

EXECUÇÃO

Agent

- artefatos aplicáveis
- sistema e tools
- captura operacional

↓

comparação

→ Evidence Record

Para manter o exemplo reproduzível, o corpus utilizado neste estudo segue os mesmos paths definidos nas seções anteriores:

```
.cursor/
├─ rules/
│   └─ persistir-antes-de-notificar.mdc
├─ agents/
│   └─ backend-development.md
├─ skills/
│   └─ validate-api-contract/
│       ├── SKILL.md
│       └─ contracts/
│           └─ report-request.contract.yaml
├─ playbooks/
│   └─ operacao-assincrona.md
├─ knowledge/
│   └─ signalr-operacoes-assincronas.md
└─ evals/
    └─ eval-async-report.yaml
```

Essa estrutura também preserva o ownership definido anteriormente: o Contract utilizado exclusivamente por validate-api-contract permanece dentro da própria Skill.

3.8.1 Eval Spec do caso

A Eval Spec precisa existir **antes da execução**. Ela deriva das Assertions aprovadas e define quais artefatos são esperados no cenário, quais propriedades precisam ser satisfeitas e quais verificadores produzirão os resultados.

```
schemaVersion: 1.0.0
id: EVAL-ASYNC-REPORT-001
version: 1.0.0
name: concluir-relatorio-reconciliar-tela

agentUnderEvaluation:
  name: backend-development
  path: .cursor/agents/backend-development.md

query: >
  O relatório pode terminar depois que o cliente se desconectar.
  Proponha o contrato HTTP e o fluxo assíncrono necessários para
  atualizar a tela com segurança e permitir que o resultado
  continue consultável posteriormente.
```

```

expected:
  shouldUse:
    - .cursor/playbooks/operacao-assincrona.md
    - .cursor/skills/validate-api-contract/SKILL.md
    - .cursor/skills/validate-api-contract/contracts/report-request.contract.yaml
    - .cursor/rules/persistir-antes-de-notificar.mdc

  mustSatisfy:
    - ASSERT-REPORT-001
    - ASSERT-REPORT-002
    - ASSERT-REPORT-003

evaluationMethod: hybrid

verification:
  artifactParticipation:
    verifier: retrievalTrace

  assertions:
    ASSERT-REPORT-001:
      verifier: causalOrderCheck

    ASSERT-REPORT-002:
      verifier: statusApiCheck

    ASSERT-REPORT-003:
      verifier: approvedLlmJudge
      rubricId: RUBRIC-ASYNC-CLAIMS-001
      rubricVersion: 1.0.0

```

`shouldUse` registra uma expectativa de participação; não afirma antecipadamente que esses artefatos serão observados durante a execução.

`mustSatisfy` aponta para Assertions já aprovadas. A Eval não redefine seu significado depois de observar a resposta.

O Eval Runner executa `.cursor/agents/backend-development.md`, captura a participação observável dos artefatos, resposta, tools, testes, traces e o estado operacional. Os verificadores processam esses dados e produzem resultados que serão consolidados pelo Evidence Record.

3.8.2 Evidence Record do caso

Com esperado e observado disponíveis, o Evidence Recorder preserva a instância final da avaliação:

```

{
  "schemaVersion": "1.0.0",
  "evidenceId": "EV-ASYNC-REPORT-001",
  "requestId": "REQ-ASYNC-REPORT-001",
  "correlationId": "corr-9812",
  "eval": {
    "id": "EVAL-ASYNC-REPORT-001",
    "version": "1.0.0"
  },
  "expected": {
    "artifacts": [
      ".cursor/playbooks/operacao-assincrona.md",
      ".cursor/skills/validate-api-contract/SKILL.md",
      ".cursor/skills/validate-api-contract/contracts/report-request.contract.yaml",

```

```

    ".cursor/rules/persistir-antes-de-notificar.mdc"
  ],
  "assertions": [
    "ASSERT-REPORT-001",
    "ASSERT-REPORT-002",
    "ASSERT-REPORT-003"
  ]
},
"observed": {
  "artifacts": [
    {
      "path": ".cursor/playbooks/operacao-assincrona.md",
      "evidence": "retrieval-trace",
      "result": "PASS"
    },
    {
      "path": ".cursor/skills/validate-api-contract/SKILL.md",
      "evidence": "retrieval-trace",
      "result": "PASS"
    },
    {
      "path": ".cursor/skills/validate-api-contract/contracts/report-request.contract.yaml",
      "evidence": "retrieval-trace",
      "result": "PASS"
    },
    {
      "path": ".cursor/rules/persistir-antes-de-notificar.mdc",
      "evidence": "retrieval-trace",
      "result": "PASS"
    }
  ],
  "assertions": [
    {
      "id": "ASSERT-REPORT-001",
      "verifier": "causalOrderCheck",
      "result": "PASS"
    },
    {
      "id": "ASSERT-REPORT-002",
      "verifier": "statusApiCheck",
      "result": "PASS"
    },
    {
      "id": "ASSERT-REPORT-003",
      "verifier": "approvedLlmJudge",
      "result": "PASS"
    }
  ],
  "runtime": {
    "durableStatus": "completed",
    "persistedSequence": 451,
    "notificationPublishedSequence": 452,
    "statusApi": "consistent"
  }
},
"verdict": {
  "functional": "PASS",

```

```

    "lineage": "PASS",
    "overall": "PASS"
  },
  "tests": [
    {
      "id": "TEST-E2E-REPORT-007",
      "result": "PASS"
    }
  ],
  "residualRisks": []
}

```

O Evidence Record não inventa fatos que não estavam disponíveis durante a execução. Neste exemplo, a ordem causal pode ser sustentada porque a captura operacional registra:

```

persistência: 451
notificação: 452

```

e a participação dos artefatos é registrada apenas porque existe evidência correspondente em retrieval-trace.

O ciclo completo do estudo pode então ser reconstruído:

```

requisito
  ↓
Rule + Contract
  ↓
Assertions
  ↓
Eval Spec
  ↓
Agent + artefatos aplicáveis
  ↓
execução
  ↓
captura operacional
  ↓
verificadores
  ↓
Evidence Record

```

O caso não está fechado apenas porque a resposta parece correta. Ele está fechado quando existe evidência suficiente para sustentar os critérios exigidos pela Eval.

Se um comportamento funcional estiver comprovado, mas a telemetria não permitir confirmar a participação de determinado artefato, preserve as duas dimensões:

```

Resultado funcional: PASS
Lineage do artefato: INCONCLUSIVE

```

A falta de observabilidade de um elo não deve ser convertida automaticamente em FAIL. Da mesma forma, uma resposta funcionalmente correta não deve ser utilizada como prova retroativa de que aquele artefato participou da execução.

Capítulo 4

Laboratório 3 — kits e templates práticos

Esta parte entrega dois laboratórios que funcionam em conjunto. Os arquivos completos acompanham este e-book na pasta `lab/`; os blocos seguintes apresentam apenas a estrutura, o papel e o ponto de entrada de cada conjunto.

4.1 Kit A - contexto sob avaliação

O Kit A contém o contexto utilizado pelo Backend Development Agent no caso assíncrono e os arquivos necessários para executar a avaliação.

O corpus efetivamente disponibilizado ao Agent permanece em `lab/.cursor/`. Fixtures, Evals e Evidence Records ficam fora desse diretório para evitar que resultados de avaliações anteriores contaminem a execução que está sendo medida.

```
lab/
├── .cursor/
│   ├── README.md
│   ├── rules/
│   │   └── persistir-antes-de-notificar.mdc
│   ├── agents/
│   │   └── backend-development.md
│   ├── skills/
│   │   ├── validate-api-contract/
│   │   │   ├── SKILL.md
│   │   │   ├── contracts/
│   │   │   │   └── report-request.contract.yaml
│   │   │   ├── references/
│   │   │   │   └── http-contract-guidance.md
│   │   │   └── scripts/
│   │   │       └── validate_contract.py
│   ├── playbooks/
│   │   └── operacao-assincrona.md
│   ├── knowledge/
│   │   └── signalr-operacoes-assincronas.md
├── evals/
│   └── eval-async-report.yaml
├── fixtures/
│   └── async-report-observed.json
├── schemas/
│   └── evidence-record.schema.json
└── evidence/
    └── EV-ASYNC-REPORT-001.json
```

A separação preserva duas responsabilidades:

```
.cursor/
→ contexto disponível para a execução

evals/ + fixtures/ + schemas/ + evidence/
→ avaliação e registro do comportamento observado
```

4.1.1 Ordem de execução

1. Abra o repositório no Cursor e envie ao Backend Development Agent a query definida em `evals/eval-async-report.yaml`.
2. Registre os artefatos cuja participação puder ser observada; preserve também a resposta e as tools executadas.
3. Execute o validador determinístico do Contract.
4. Grave a captura operacional em `fixtures/async-report-observed.json`.
5. Passe a Eval Spec e a captura observada ao Kit B.
6. Valide o Evidence Record produzido contra `schemas/evidence-record.schema.json` antes de aceitar o veredito.

4.1.2 Definition of Done do Kit A

- o Agent, o Playbook, a Skill, o Knowledge e os demais artefatos utilizam identidade e metadados compatíveis com seus respectivos formatos;
- a Cursor Project Rule canônica está em `.cursor/rules/persistir-antes-de-notificar.mdc`;
- o Contract exclusivo da Skill permanece em `.cursor/skills/validate-api-contract/contracts/`;
- a Skill possui `description` testável e referencia apenas recursos realmente existentes;
- o validador determinístico consegue verificar as condições do Contract previstas pelo laboratório;
- a Eval Spec define o esperado antes da execução;
- o caso demonstra persistência confirmada antes da notificação e reconciliação posterior pela API;
- o Evidence Record produzido valida contra o schema do laboratório;
- qualquer critério funcional ou de lineage sem evidência suficiente permanece `INCONCLUSIVE`, sem transformar ausência de observabilidade em `FAIL`.

4.2 Kit B - harness de avaliação

O Kit B recebe os fatos coletados pelo Eval Runner e os transforma em uma avaliação reproduzível. Ele separa coordenação da avaliação, capacidade especializada de auditoria, critérios semânticos, validação estrutural e registro de evidência.

Ele não substitui nem reexecuta o Backend Development Agent, que continua sendo o agente sob avaliação. O fluxo é:

```
Backend Development Agent
→ execução

Eval Runner
→ coleta fatos observáveis

Context Evaluation Agent
→ coordena a avaliação

audit-context-execution
→ aplica critérios e consolida resultados
```

Evidence Record

→ preserva expected, observed e verdict

A estrutura do laboratório é:

```
evaluation/
├── .cursor/
│   ├── agents/
│   │   └── context-evaluation.md
│   └── skills/
│       ├── audit-context-execution/
│       │   ├── SKILL.md
│       │   ├── references/
│       │   │   └── semantic-criteria.md
│       │   └── scripts/
│       │       └── validate_evidence.py
├── evals/
│   └── assertion-variations.yaml
├── schemas/
│   └── evidence-record.schema.json
└── evidence/
    └── EV-ASYNC-REPORT-001.json
```

A separação preserva responsabilidades diferentes:

- `.cursor/agents/` contém o Agent que coordena a avaliação;
- `.cursor/skills/` contém a capacidade especializada de auditoria;
- `references/` contém critérios consultados pela Skill quando aplicáveis;
- `scripts/` contém verificações determinísticas auxiliares;
- `evals/` preserva critérios e variações utilizadas na avaliação;
- `schemas/` define a estrutura válida do Evidence Record;
- `evidence/` recebe somente registros produzidos a partir de execuções identificadas.

O Kit B deve trabalhar apenas com o esperado previamente definido e com fatos observados fornecidos pelo Eval Runner. Ele não deve inventar artefatos recuperados, tools executadas, testes, transições de estado ou evidências ausentes.

Quando um critério não puder ser confirmado nem negado com os dados disponíveis, o resultado correspondente permanece INCONCLUSIVE.

Assim, os dois kits mantêm responsabilidades complementares:

Kit A

→ executa o contexto sob avaliação
→ produz fatos observáveis

Kit B

→ compara esperado e observado
→ aplica verificadores
→ produz e valida o Evidence Record

4.2.1 Agent de avaliação preenchido

O arquivo `.cursor/agents/context-evaluation.md` coordena a avaliação depois que o Eval Runner já executou o caso e coletou os fatos observáveis. Ele não reexecuta o agente sob avaliação nem substitui os verificadores definidos pela Eval Spec.

```
---
name: context-evaluation
description: Coordena a avaliação de uma execução observada contra uma Eval Spec versionada.
```

```
---

# Context Evaluation Agent

## Missão

- Coordenar uma avaliação reproduzível.
- Comparar expectativas previamente definidas com fatos observados.
- Preservar separadamente resultado funcional e lineage.

## Entradas obrigatórias

- Eval Spec e Assertions aplicáveis.
- Identificação do agente sob avaliação.
- Corpus snapshot, instructions version e model snapshot.
- Captura observada produzida pelo Eval Runner.
- Resultados dos verificadores determinísticos já executados.
- Schema do Evidence Record.

## Roteamento

1. Valide se as entradas obrigatórias estão identificadas.
2. Preserve os resultados dos verificadores determinísticos.
3. Utilize a Skill `audit-context-execution` para os critérios aplicáveis.
4. Utilize julgamento semântico somente quando a Eval Spec exigir.
5. Produza o Evidence Record segundo o schema fornecido.

## Restrições

- NÃO reexecute o agente sob avaliação.
- NÃO invente artefatos recuperados, tools, testes, traces ou transições.
- NÃO substitua resultado determinístico por julgamento semântico.
- NÃO use uma resposta correta como prova de participação de um artefato.
- NÃO altere retrospectivamente as expectativas da Eval Spec.

## Condições de parada

- PARE se a Eval Spec ou o schema necessário não estiver disponível.
- PARE se uma Assertion normativa não possuir fonte aprovada identificável.
- MARQUE `INCONCLUSIVE` quando a evidência disponível não permitir confirmar nem negar um critério.
- PRESERVE avaliações funcionais válidas mesmo quando lineage permanecer `INCONCLUSIVE`.

## Evidência mínima

- Registre versões e identificadores necessários para reproduzir a avaliação.
- Registre somente participação de artefatos sustentada por evidência observável.
- Preserve cada Assertion, seu verificador e respectivo resultado.
- Preserve separadamente `functional`, `lineage` e `overall`.
- Registre limitações de observabilidade sem convertê-las automaticamente em `FAIL`.
```

Os campos de governança adicionais, como id, owner, status, version e temperature, podem permanecer no catálogo adotado pelo laboratório. O frontmatter operacional do Agent fica restrito aos campos utilizados para sua identificação e descoberta pelo host.

4.2.2 Skill preenchida

A Skill `audit-context-execution` executa a capacidade especializada de auditoria utilizada pelo Context Evaluation Agent. Ela recebe uma execução já capturada, preserva os resultados determinísticos disponíveis, avalia somente os critérios semânticos previstos pela Eval Spec e produz um Evidence Record estruturado.

```

---
name: audit-context-execution
description: >
  Audita uma execução já capturada, compara Assertions esperadas com fatos
  observados e produz um Evidence Record. Use após o Eval Runner concluir
  a execução e disponibilizar os fatos e resultados determinísticos observáveis.
metadata:
  display-name: "Auditar Execução de Contexto"
  owner: "context-curation"
  version: "0.1.0"
---

# Auditar Execução de Contexto

## Quando usar

- Depois que o Eval Runner capturar resposta, artefatos observáveis, tools, traces, testes e estado.
- Para comparar `expected` e `observed` segundo uma Eval Spec previamente definida.
- Para avaliar critérios semânticos que não possam ser decididos pelos verificadores determinísticos disponíveis.
- Para produzir e validar o Evidence Record da execução.

## Quando não usar

- Não use para executar novamente o agente sob avaliação.
- Não use para criar ou alterar retrospectivamente Assertions.
- Não use para inventar artefatos, tools, testes, traces ou estados ausentes.
- Não use julgamento semântico para substituir um resultado determinístico válido.

## Entradas e pré-condições

- Eval Spec e Assertions aplicáveis.
- Identificação da execução e do agente sob avaliação.
- Fatos capturados pelo Eval Runner.
- Resultados dos verificadores determinísticos disponíveis.
- Schema do Evidence Record.

## Fluxo principal

1. Validar presença e identificação das entradas obrigatórias.
2. Separar explicitamente `expected` de `observed`.
3. Preservar os resultados dos verificadores determinísticos já executados.
4. Consultar `references/semantic-criteria.md` somente quando houver critério semântico aplicável.
5. Avaliar semanticamente apenas os critérios indicados pela Eval Spec.
6. Classificar cada Assertion como `PASS`, `FAIL` ou `INCONCLUSIVE`.
7. Avaliar separadamente resultado funcional e lineage.
8. Produzir o Evidence Record.
9. Executar `scripts/validate_evidence.py` para validar sua estrutura antes da conclusão.

```

Validações determinísticas

- Preserve resultados determinísticos fornecidos pelo Eval Runner ou harness.
- Verifique IDs, campos obrigatórios e estrutura do Evidence Record.
- Não registre fonte, tool, teste ou artefato como observado sem evidência correspondente.
- Quando a participação esperada de um artefato não puder ser confirmada nem negada, marque o critério de lineage correspondente como ``INCONCLUSIVE``.
- Use ``FAIL`` somente quando a evidência contradisser o critério ou demonstrar suficientemente que o comportamento esperado não ocorreu.

Critérios semânticos

Consulte ``references/semantic-criteria.md`` somente para Assertions cuja verificação exija interpretação semântica.

- Avalie equivalência de sentido, contradição e adequação conforme o critério aprovado.
- Avalie proibições por seu comportamento esperado correspondente, inclusive quando aparecerem por paráfrase.
- Registre o trecho observado, o critério aplicado e o resultado.
- Não registre cadeia de pensamento ou justificativa interna do modelo.

Guardrails e condições de parada

- NÃO invente artefato, resultado, trace, teste ou transição ausente.
- NÃO transforme ausência de evidência em ``PASS`` ou ``FAIL``.
- NÃO use uma resposta funcionalmente correta como prova de participação de um artefato.
- NÃO altere ``expected`` depois de observar o resultado.
- MARQUE ``INCONCLUSIVE`` somente nos critérios que não puderem ser confirmados nem negados.
- CONTINUE avaliando critérios independentes quando houver evidência suficiente para eles.
- PARE a avaliação completa somente quando faltar uma entrada indispensável para executar o processo com segurança.

Saída e Definition of Done

- Evidence Record válido contra o schema fornecido.
- Cada Assertion contém identificador, verificador e resultado.
- ``functional``, ``lineage`` e ``overall`` permanecem separados.
- Evidências ausentes e limitações de observabilidade são registradas explicitamente.
- A camada a investigar é indicada somente quando os fatos disponíveis sustentarem essa associação.
- A próxima ação é registrada somente quando puder ser derivada das evidências disponíveis.

Evidência e checklist final

- [] Agente, corpus, Eval Spec, instructions e modelo estão identificados nas versões ou snapshots necessários.
- [] ``expected`` foi preservado separadamente de ``observed``.
- [] Resultados determinísticos foram preservados sem substituição por julgamento semântico.
- [] Julgamento semântico foi utilizado somente nos critérios previstos.
- [] Artefatos observados possuem evidência correspondente.
- [] Evidências ausentes estão declaradas explicitamente.
- [] ``PASS``, ``FAIL`` e ``INCONCLUSIVE`` foram aplicados por critério.
- [] Resultado funcional e lineage permanecem separados.
- [] O Evidence Record foi validado por ``scripts/validate_evidence.py``.
- [] O veredito pode ser reconstruído a partir dos fatos registrados.

O frontmatter utiliza name e description obrigatórios e mantém os dados adicionais dentro de

metadata, que a especificação Agent Skills permite como mapa de propriedades adicionais. A Skill também referencia seus recursos por paths relativos ao próprio diretório, conforme a convenção do formato. [^agent-skills-spec]

4.2.3 Variações de Assertions

O YAML abaixo usa a estrutura nativa de dados; por isso, não recebe títulos # ou ##.

Os exemplos também reforçam uma distinção importante: uma Assertion deve utilizar o verificador mais adequado ao comportamento que pretende observar. A Eval pode ser **híbrida** porque combina Assertions determinísticas e semânticas; isso não significa que toda Assertion precise executar os dois tipos de verificação.

```
evaluationMethod: hybrid

assertions:
  - id: ASSERT-EVIDENCE-001
    type: requiredField
    sourceOriginal: SRC-020
    sourceCurrent: schemas/evidence-record.schema.json#/required
    statement: "O Evidence Record deve conter evidenceId."
    evaluationMethod: deterministic
    verifyWith: jsonSchema

  - id: ASSERT-SIGNALR-002
    type: requiredBehavior
    sourceOriginal: SRC-016
    sourceCurrent: .cursor/rules/persistir-antes-de-notificar.mdc#proibicao
    statement: "A solução não deve tratar SignalR como fonte durável do estado."
    evaluationMethod: semantic
    verifyWith: approvedLlmJudge

  - id: ASSERT-ORDER-003
    type: requiredBehavior
    sourceOriginal: SRC-014
    sourceCurrent: .cursor/rules/persistir-antes-de-notificar.mdc#obrigacao
    statement: "A confirmação da persistência deve preceder a notificação de conclusão."
    evaluationMethod: deterministic
    verifyWith: causalOrderCheck
```

No terceiro caso, causalOrderCheck é suficiente quando o trace torna a ordem observável. Um LLM Judge adicional não melhora automaticamente a avaliação e só deve ser utilizado quando existir um critério semântico que o verificador determinístico não consiga decidir.

A combinação dos métodos ocorre no nível da Eval:

```
ASSERT-EVIDENCE-001
→ determinístico

ASSERT-SIGNALR-002
→ semântico

ASSERT-ORDER-003
→ determinístico

conjunto da Eval
→ híbrido
```

4.2.4 Execução do kit

1. Copie as estruturas dos Kits A e B para o diretório de laboratório.
2. Identifique e preserve versões, snapshots, fontes e Assertions antes da execução.
3. Execute a query definida pela Eval Spec contra o agente sob avaliação.
4. Grave a captura bruta produzida pelo Eval Runner em área com retenção adequada.
5. Execute os verificadores determinísticos previstos pela Eval Spec.
6. Passe a Eval Spec, a captura observada e os resultados determinísticos ao Context Evaluation Agent.
7. Utilize audit-context-execution somente para os critérios aplicáveis e para a produção do Evidence Record.
8. Valide o Evidence Record por código contra o schema definido pelo laboratório.
9. Compare a nova execução com a baseline utilizando os mesmos casos e critérios versionados.

A comparação deve preservar a ordem:

```
expected
→ execução
→ observed
→ verificadores
→ verdict
```

Não altere uma expectativa retrospectivamente para fazê-la coincidir com o resultado observado. Se o comportamento esperado realmente precisar mudar, versione a expectativa e execute novamente o caso.

→ O Context Evaluation Agent coordena a avaliação; a Skill audit-context-execution audita uma execução já observada; a Eval Spec preserva o esperado; os verificadores produzem resultados por critério; o Evidence Record registra fatos, resultados, limitações e veredito.

4.3 Template mínimo de Agent

Use este template para definir um Agent especializado em `.cursor/agents/<nome>.md`. O frontmatter operacional permanece enxuto; informações adicionais de governança, como ID, owner, versão ou temperatura, podem ser mantidas no catálogo adotado pelo projeto.

```
---
name: <nome-kebab-case>
description: <o que este agente coordena e quando deve ser utilizado>
---

# <Nome legível do agente>

## Missão

<resultado principal que este agente deve coordenar>

## Invariantes

- DEVE <comportamento obrigatório>.
- NÃO DEVE <comportamento proibido>.

## Roteamento

- SE <sinal de aplicabilidade>, USE <Playbook ou Skill>.
- SE <informação necessária não estiver disponível>, PARE e solicite o dado necessário.

## Condições de parada
```

- PARE quando **<critério material impedir uma decisão segura>**.
- NÃO prossiga quando <fonte, Contract ou critério necessário estiver ausente e não puder ser recuperado>.

Evidência mínima

- REGISTRE <resultado, identificador ou evidência observável necessária>.

O Agent deve permanecer pequeno. Sua função principal é **decidir, rotear, preservar invariantes e reconhecer quando parar**; detalhes especializados devem ser recuperados por Skills, Playbooks e outros artefatos aplicáveis.

4.4 Template mínimo de Skill

Uma Skill representa uma capacidade especializada, reutilizável e com limites claros. O nome deve utilizar kebab-case e corresponder ao nome do diretório da Skill.

```
---
name: <nome-kebab-case-igual-ao-diretorio>
description: >
  <o que a Skill faz, quando usar e termos suficientes para descoberta>
metadata:
  display-name: "<Nome legível>"
  owner: "<time>"
  version: "0.1.0"
---

# <Nome legível>

## Quando usar

- <situação em que esta capacidade é aplicável>.

## Quando não usar

- <tarefa vizinha ou condição fora do escopo>.

## Entradas e pré-condições

- <entrada verificável necessária para execução>.

## Fluxo principal

1. <inspecionar>.
2. <executar>.
3. <validar>.

## Guardrails e condições de parada

- NÃO <comportamento proibido>.
- PARE quando <informação ou evidência material estiver ausente>.

## Recursos empacotados

- `references/`: <conteúdo consultado quando necessário>.
- `scripts/`: <execução determinística quando necessária>.
```

```
- `assets/`: <template ou recurso utilizado pela Skill>.

## Saída e Definition of Done

- <resultado observável esperado>.
- <validação necessária para considerar a execução concluída>.

## Evidência e checklist final

- [ ] <comando, arquivo ou resultado observável registrado>.
```

references/, scripts/ e assets/ são recursos opcionais. Inclua apenas diretórios realmente necessários à capacidade; a existência dessas pastas não faz parte da Definition of Done de toda Skill.

Comece pelo template mínimo. Acrescente referências, scripts, critérios, perguntas ou guardrails somente quando a complexidade da capacidade ou os Evals demonstrarem necessidade.

4.5 Anatomia ampliada opcional de uma Skill

? Convenção interna, não extensão obrigatória do padrão

A especificação Agent Skills define o formato de SKILL.md, incluindo name e description obrigatórios, e permite que o corpo Markdown seja organizado livremente. scripts/, references/ e assets/ são recursos opcionais carregados conforme necessário.[^agent-skills-spec]

A estrutura abaixo é uma proposta de curadoria deste livro para equipes que precisam operar Skills com regras, critérios, perguntas, evidências e conteúdo progressivo. Adote-a somente quando melhorar execução, manutenção ou recuperabilidade, e valide a Skill no cliente realmente utilizado.

Avaliação da proposta. A anatomia é útil porque reúne descoberta, escopo, guardrails e verificação sem exigir que todo o conhecimento permaneça dentro de SKILL.md. Quatro cuidados evitam transformar a Skill em um novo arquivo monolítico:

- a description deve conter informações suficientes para descoberta; sinais no corpo servem para confirmar aplicabilidade depois que a Skill já entrou em uso;
- Rules compartilhadas devem ser referenciadas por seus paths canônicos, em vez de copiadas para cada Skill;
- perguntas devem seguir **descobrir → verificar → perguntar**: não pergunte ao operador o que puder ser comprovado no repositório;
- decisões que variam por ambiente não devem ser congeladas no conteúdo principal sem apontar para a fonte vigente.

Frontmatter interoperável e título humano:

```
---
name: kubernetes-manifests
description: >
  Cria ou revisa manifests Kubernetes para APIs, workers e funções.
  Use quando a tarefa envolver Deployment, Service, HPA, env ou secrets.
compatibility: Requer acesso ao repositório e ao validador YAML aprovado pelo time.
metadata:
  display-name: "Kubernetes Manifests"
  owner: "platform-engineering"
  version: "1.0.0"
  book-temperature: "warm"
---
```

Kubernetes Manifests

O nome em kebab-case identifica a Skill e deve corresponder ao nome de seu diretório. # Kubernetes Manifests é o título destinado à leitura humana.

metadata aceita propriedades adicionais. Neste livro, book-temperature deixa explícito que a classificação pertence à ontologia adotada pelo projeto, e não ao conjunto de campos com semântica própria definido pela especificação Agent Skills.[^agent-skills-spec]

4.5.1 Corpo recomendado

- **Quando usar:** delimite tarefas cobertas e resultado esperado.
- **Quando não usar:** exclua tarefas vizinhas ou que exigem outra capacidade.
- **Entradas e pré-condições:** declare dados, arquivos, acessos e ferramentas necessários.
- **Sinais de aplicabilidade:** registre situações e exemplos que confirmam se a Skill se aplica depois de descoberta.
- **Fluxo principal:** descreva uma sequência curta de inspeção, decisão, execução e validação.
- **Guardrails:** referencie Rules NORMATIVE, proibições, permissões e condições de parada.
- **Recursos empacotados:** liste somente diretórios e arquivos que realmente existem.
- **Catálogo:** informe quando consultar cada referência sem carregá-la antecipadamente.
- **Receitas compactas:** registre decisões recorrentes em poucas linhas e aponte a fonte detalhada.
- **Avisos de saída:** defina mensagens condicionais que precisam chegar ao usuário.
- **Perguntas e condições de parada:** pergunte somente o que não puder ser descoberto ou verificado.
- **Saída e Definition of Done:** especifique arquivos, validações e estado observável de conclusão.
- **Evidência:** indique comandos, relatórios e identificadores que sustentam a execução.
- **Checklist final:** confirme invariantes antes da conclusão.

4.5.2 Exemplo compacto do corpo

Kubernetes Manifests

Quando usar

Use para criar ou revisar manifests de API, worker ou função quando o repositório possuir contrato de implantação identificável.

Quando não usar

- Não altere ``ingress/**`` sem autorização explícita.
- Não invente namespace, secret provider, portas ou limites de HPA.

Entradas e pré-condições

- Repositório e ambiente-alvo.
- Contrato de portas e variáveis.
- Rules aplicáveis para disponibilidade e autoscaling.

Sinais de aplicabilidade

- Pedido para criar Deployment, Service ou HPA.
- Erro de env, secret, namespace ou porta em manifest existente.
- Não se aplica a diagnóstico exclusivamente de código C# sem impacto no deployment.

Guardrails - NORMATIVE

- APLIQUE ``.cursor/rules/k8s-naming.mdc`` quando houver regra de nomenclatura aplicável.
- APLIQUE ``.cursor/rules/k8s-secrets.mdc``; nunca grave segredo em texto puro.
- PARE se ambiente, namespace ou contrato de portas não puder ser verificado.

Recursos empacotados

Esta Skill utiliza ``.SKILL.md`` e ``.references/``. Não possui ``.scripts/`` nem ``.assets/``. Não presuma recursos que não aparecem neste catálogo.

Catálogo progressivo

| Etapas | Access | Authority | Abrir quando | Path |
|------------|------------|--------------|-----------------------------|---|
| ACTIVATION | NAVIGATION | NORMATIVE | Após a Skill entrar em uso. | <code>`.SKILL.md`</code> |
| ON_DEMAND | LOOKUP | NORMATIVE | Alterar env ou secret. | <code>`.references/env-secrets.md`</code> |
| ON_DEMAND | LOOKUP | ADVISORY | Configurar HPA. | <code>`.references/hpa.md`</code> |
| DEEP_DIVE | DEEPDIVE | ILLUSTRATIVE | Investigar legado. | <code>`.references/migrations.md`</code> |

Receitas compactas

****HPA:**** use ``.autoscaling/v2``; obtenha métrica e alvo na Rule aplicável.

Detalhes: ``.references/hpa.md``.

****Env e secrets:**** derive nomes da convenção aprovada e valide referências.

Detalhes: ``.references/env-secrets.md``.

Avisos obrigatórios de saída

- SE um nome for derivado automaticamente, informe origem e transformação.
- SE houver endpoint HTTP, informe quando o Ingress estiver fora do escopo.

Perguntas e condições de parada

1. Descubra no repositório se o workload expõe HTTP.
2. Verifique se o tipo é API, worker ou função.
3. Pergunte pelo ambiente somente se não houver evidência confiável.
4. Pare antes de editar quando porta, namespace ou secret provider divergirem.

Saída e Definition of Done

- Manifests alterados somente no escopo autorizado.
- Referências entre recursos permanecem consistentes.
- Validação YAML e policy checks executados com sucesso.
- Nenhum segredo armazenado em texto puro.

Evidência e checklist final

- [] Nome derivado da convenção e Rule aplicável registrada.
- [] Deployment, Service e HPA coerentes quando aplicáveis.
- [] Portas e probes confirmadas por fonte do repositório.
- [] Referências adicionais abertas somente quando aplicáveis.
- [] Comandos, códigos de saída e arquivos alterados registrados.
- [] Avisos condicionais incluídos na resposta.

O catálogo interno evita reutilizar HOT/WARM/COLD para descrever arquivos dentro de uma Skill já classificada como WARM. Aqui, *Etapa* responde especificamente **quando um recurso interno é carregado depois que a Skill entra em uso**; *Access* descreve como chegar ao conteúdo; *Authority* indica sua força quando aplicável.

Assim, a classificação global da Skill e o carregamento de seus recursos internos permanecem dimensões distintas.

! MCP não interpreta frontmatter arbitrário

Metadados de uma Skill podem alimentar catálogos ou roteadores construídos pela equipe, mas o protocolo MCP não atribui automaticamente significado operacional a campos particulares de `SKILL.md`.

Em uma integração MCP, alinhe a descoberta aos elementos realmente expostos pelo servidor — como nomes, descrições e schemas de tools e as propriedades dos resources — e valide no cliente como esses elementos são apresentados e incorporados ao contexto.^[^mcp-routing]

4.6 Template de Playbook

Use este template para procedimentos coordenados que organizam etapas, dependências e decisões entre artefatos. O Playbook pertence à ontologia deste livro; seus metadados só possuem efeito operacional quando algum Agent, catálogo ou harness os interpreta.

```
---
id: PLAYBOOK-<DOMINIO>-<NÚMERO>
name: <nome-kebab-case>
type: playbook
description: <quando este procedimento coordenado se aplica>
temperature: warm
applicability_signals:
  - <sinal observável de aplicabilidade>
owner: <time>
status: draft
version: 0.1.0
---

# <Nome legível do Playbook>

## Quando usar

- <situação em que o procedimento é aplicável>.

## Pré-condições

- <condição necessária antes da execução>.

## Entradas

- <entrada identificada e verificável>.

## Procedimento

1. <primeira etapa>.
2. <segunda etapa>.

## Decisões condicionais
```

```
- SE <condição>, ENTÃO <ação>.
```

```
## Fallbacks
```

```
- SE <falha>, PRESERVE <estado seguro ou caminho de recuperação>.
```

```
## Condições de parada
```

```
- PARE quando <decisão, informação ou evidência necessária estiver ausente>.
```

```
## Saídas
```

```
- <resultado observável produzido>.
```

```
## Evidências
```

```
- <log, teste, trace, estado ou outro fato observável>.
```

```
## Artefatos relacionados
```

```
- Rule: `<path-da-rule>`.
- Contract: `<path-do-contract>`.
- Skill: `<path-da-skill>`.
- Eval: `<path-da-eval>`, quando aplicável.
```

O Playbook coordena a sequência, mas não deve duplicar o conteúdo canônico dos artefatos relacionados. Quando uma Rule, Contract ou Skill já possuir a informação necessária, referencie-a e mantenha no Playbook apenas o contexto suficiente para coordenar sua utilização.

4.7 Template de Rule

Neste livro, Domain Rule e Engineering Rule são classificações semânticas. Quando a Rule for executável pelo Cursor, materialize-a como uma Cursor Project Rule em `.cursor/rules/<nome-kebab-case>.mdc`.

```
---
description: >
  <quando esta Rule é aplicável e qual comportamento obrigatório ela protege>
globs:
alwaysApply: false
---

# <Nome legível da Rule>

**Classificação semântica:** <Domain Rule | Engineering Rule>

## Obrigação

- DEVE <comportamento positivo>.

## Proibição

- NÃO DEVE <atalho ou comportamento proibido>.

## Fallback
```

```
- SE <falha>, PRESERVE <estado ou comportamento seguro>.
```

```
## Evidência
```

```
- REGISTRE <fato observável necessário para verificação>.
```

A autoridade da Rule vem da fonte e da política de governança aplicável. Campos organizacionais adicionais, como owner, versão ou temperatura, pertencem ao catálogo do projeto e não precisam ser inseridos no frontmatter operacional da Cursor Project Rule.

4.8 Template de Contract

O Contract torna condições relevantes observáveis e verificáveis. Quando pertencer exclusivamente a uma Skill, mantenha-o dentro do diretório dessa Skill.

```
id: CONTRACT-<DOMINIO>-<NÚMERO>
name: <nome-kebab-case-da-operacao>
operation: <identificador-da-operacao>
```

```
input:
  required: []
```

```
output:
  success: {}
```

```
preconditions: []
```

```
postconditions:
  accepted: []
  completed: []
```

```
forbidden: []
```

```
observability:
  correlationFields: []
```

Separe accepted de completed quando a operação possuir momentos distintos de aceitação e conclusão.

4.9 Template de Eval

A Eval Spec define o esperado **antes da execução**. Ela aponta para Assertions previamente aprovadas e associa cada critério ao verificador adequado.

```
schemaVersion: 1.0.0
id: EVAL-<DOMINIO>-<NÚMERO>
version: 1.0.0
name: <nome-kebab-case-do-eval>

agentUnderEvaluation:
  name: <nome-do-agent>
  path: .cursor/agents/<nome-do-agent>.md

corpus:
  snapshot: <versão>

instructions:
```

```

version: <versão>

model:
  snapshot: <identificador>

query: >
  <pergunta ou tarefa>

expected:
  shouldUse:
    - <path-do-artefato>

  mustSatisfy:
    - ASSERT-<DOMINIO>-<NÚMERO>

evaluationMethod: deterministic|semantic|hybrid

verification:
  artifactParticipation:
    verifier: <verificador-de-participacao>

  assertions:
    ASSERT-<DOMINIO>-<NÚMERO>:
      verifier: <verificador-aplicável>

```

shouldUse representa participação esperada, não participação comprovada. mustSatisfy aponta para comportamentos esperados já definidos pelas Assertions.

4.10 Template de Evidence Record

O Evidence Record preserva a comparação entre esperado e observado. Resultado funcional e lineage permanecem separados para que ausência de observabilidade não seja confundida com falha funcional.

```

schemaVersion: 1.0.0
evidenceId: EV-<ANO>-<NÚMERO>
requestId: REQ-<ANO>-<NÚMERO>
correlationId: <correlation-id>

eval:
  id: EVAL-<DOMINIO>-<NÚMERO>
  version: <versão>

context:
  agent: <nome-do-agent>
  corpusSnapshot: <versão>
  instructionsVersion: <versão>
  modelSnapshot: <identificador>

expected:
  artifacts:
    - <path-esperado>
  assertions:
    - ASSERT-<DOMINIO>-<NÚMERO>

observed:
  artifacts:

```

```
- path: <path-observado>
  evidence: <trace-ou-fonte>
  result: PASS|FAIL|INCONCLUSIVE

assertions:
- id: ASSERT-<DOMINIO>-<NÚMERO>
  verifier: <verificador>
  result: PASS|FAIL|INCONCLUSIVE

verdict:
  functional: PASS|FAIL|INCONCLUSIVE
  lineage: PASS|FAIL|INCONCLUSIVE
  overall: PASS|FAIL|INCONCLUSIVE

tests: []
residualRisks: []
timestamp: <ISO-8601>
```

Registre somente fatos sustentados pela execução. Quando a evidência não permitir confirmar nem negar determinado critério, preserve INCONCLUSIVE na dimensão correspondente em vez de inferir PASS ou FAIL.

Capítulo 5

Apêndice — Laboratórios e checkpoints práticos

Os laboratórios deste apêndice complementam as seções conceituais do livro. Cada experimento detalha o que preparar, onde executar, o que observar, como registrar evidências e como interpretar os resultados.

5.1 Laboratório — Agent HOT

Este laboratório complementa a definição de **Agent HOT** e verifica duas responsabilidades:

1. **roteamento:** reconhecer uma tarefa de contrato HTTP e encaminhá-la para a capacidade adequada;
2. **condição de parada:** interromper ou solicitar informação quando um dado necessário não puder ser recuperado.

O laboratório utiliza o adapter:

```
.cursor/  
└─ agents/  
   └─ backend-development.md
```

e reutiliza a skill criada anteriormente:

```
.cursor/  
└─ skills/  
   └─ validate-api-contract/  
      ├── SKILL.md  
      └─ contracts/  
         └─ report-request.contract.yaml
```

O fluxo completo do experimento é:

```
definir expectativa  
  ↓  
executar Run 001  
  ↓  
avaliar roteamento  
  ↓  
executar Run 002  
  ↓  
avaliar condição de parada  
  ↓  
registrar evidências  
  ↓
```

interpretar o checkpoint

5.1.1 1. Registre a expectativa antes de executar

Antes das duas execuções, crie:

```
.cursor/
├─ evals/
│   └─ checkpoints/
│       └─ agent-hot/
│           └─ expected.md
```

Arquivo `.cursor/evals/checkpoints/agent-hot/expected.md`:

Agent HOT – expectativa

Run 001 – roteamento

Entrada

Uma tarefa solicita alteração de um contrato HTTP e fornece informações suficientes para que a análise prossiga.

Comportamento esperado

- o Agent reconhece que a tarefa envolve contrato HTTP;
- a capacidade ``validate-api-contract`` é considerada aplicável;
- a proposta respeita os critérios do Contract utilizado pela Skill;
- o Agent não ignora uma validação obrigatória de contrato.

Lineage esperado

backend-development → validate-api-contract

Run 002 – condição de parada

Entrada

Uma tarefa solicita uma alteração cuja nova regra de aceite não foi informada e não está disponível nas fontes do repositório.

Comportamento esperado

- o Agent identifica a informação ausente;
- solicita o critério necessário ou interrompe a execução;
- não inventa a regra de aceite;
- não apresenta uma solução final como se a informação estivesse disponível.

Esse arquivo registra **o que deve acontecer antes de observar as respostas do Agent**. Mantenha-o inalterado durante as duas rodadas.

Se descobrir posteriormente que a própria expectativa estava incorreta, preserve as execuções realizadas com a versão anterior, ajuste ou versione `expected.md` e somente então inicie uma nova rodada.

5.1.2 2. Execute o Run 001 — roteamento

Abra a raiz do repositório no Cursor e inicie **uma nova conversa no Agent principal**.

Não reutilize uma conversa anterior, porque contexto residual pode influenciar o comportamento observado.

Envie:

```
# Tarefa

## Objetivo

Revisar o contrato HTTP utilizado para iniciar a geração de um relatório que pode levar vários minutos.

## Execução

Use o subagent `backend-development` para coordenar a análise.

## Requisitos

- A operação deve responder inicialmente com `202 Accepted`.
- A resposta deve fornecer `jobId` e `statusUrl`.
- O cliente precisa conseguir recuperar o estado após uma desconexão.
- Notificação em tempo real não pode ser a fonte canônica do estado.

## Saída esperada

1. contrato HTTP proposto;
2. validações aplicadas;
3. ordem relevante das operações;
4. artefatos utilizados para sustentar a proposta.
```

O prompt informa qual subagent deve coordenar a execução, mas **não manda utilizar diretamente a Skill**.

A transição que está sendo testada é:

```
backend-development
  ↓
validate-api-contract
```

Ela deve decorrer das instruções de roteamento do próprio Agent.

5.1.3 3. Preserve o resultado do Run 001

Ao final da execução, copie a resposta produzida pelo Cursor sem corrigi-la ou reescrevê-la.

Salve em:

```
.cursor/
├─ evidence/
│   └─ checkpoints/
│       └─ agent-hot/
│           └─ run-001-response.md
```

Esse arquivo contém o **resultado bruto observado**.

A afirmação da própria resposta de que determinado arquivo ou Skill foi utilizado não deve ser tratada automaticamente como prova de lineage quando o host não fornecer evidência suficiente para confirmá-la.

5.1.4 4. Avalie o Run 001

Compare a resposta com a seção **Run 001 — roteamento** de `expected.md`.

Use estas verificações:

| Verificação | O que observar |
|--|---|
| Contrato HTTP reconhecido como parte da tarefa | A resposta efetivamente analisa ou propõe o contrato solicitado. |
| Validação especializada aplicável | A solução apresenta as verificações esperadas para o contrato. |
| Contract respeitado | 202, jobId, statusUrl, durabilidade e recuperação permanecem coerentes. |
| Skill utilizada | Existe evidência observável de participação de validate-api-contract. |

Resultado funcional e lineage devem ser avaliados separadamente.

Se a solução estiver correta, mas o host não fornecer evidência suficiente para comprovar a participação da Skill:

Resultado funcional: PASS

Lineage backend-development → validate-api-contract: INCONCLUSIVE

Se houver evidência suficiente de que o Agent ignorou o roteamento obrigatório mesmo diante de uma tarefa de contrato HTTP, registre FAIL para a transição correspondente.

5.1.5 5. Execute o Run 002 — condição de parada

Inicie **outra conversa nova no Agent principal do Cursor**. Não reutilize a conversa do Run 001.

Envie:

Tarefa

Objetivo

Alterar o contrato HTTP de geração de relatórios para atender a uma nova regra de aceite definida pelo negócio.

Execução

Use o subagent ``backend-development`` para coordenar a análise.

Contexto

A nova regra substitui um comportamento atual, mas seu critério de aceite ainda não foi informado e não está documentado nos artefatos disponíveis no repositório.

Saída esperada

Informe a alteração necessária no contrato e as decisões utilizadas para sustentá-la.

A informação ausente é proposital. O teste não procura uma solução plausível; ele verifica se o Agent reconhece que **não possui fundamento suficiente para concluir a tarefa**.

O fluxo esperado é:

critério necessário ausente

↓

não recuperável das fontes disponíveis

↓

interromper ou solicitar informação

↓

não inventar

5.1.6 6. Preserve o resultado do Run 002

Copie a resposta produzida pelo Cursor sem modificá-la e salve em:

```
.cursor/
└─ evidence/
   └─ checkpoints/
      └─ agent-hot/
         └─ run-002-response.md
```

Ao final das duas execuções, a estrutura deve conter:

```
.cursor/
└─ evidence/
   └─ checkpoints/
      └─ agent-hot/
         ├── run-001-response.md
         └─ run-002-response.md
```

5.1.7 7. Avalie o Run 002

Compare a resposta com a seção **Run 002 — condição de parada** de `expected.md`.

| Comportamento observado | Resultado |
|--|-----------|
| Solicita o critério de aceite ausente | PASS |
| Interrompe explicitamente até que o critério seja fornecido | PASS |
| Expõe a ausência da informação, mas apresenta a solução final como se fosse válida | FAIL |
| Inventa uma regra de aceite e continua | FAIL |

Neste cenário, a condição de parada é observável diretamente na resposta. O objetivo é verificar o comportamento final, e não inferir estados internos do modelo.

5.1.8 8. Registre as avaliações

Complete agora a estrutura de evidência:

```
.cursor/
└─ evidence/
   └─ checkpoints/
      └─ agent-hot/
         ├── run-001-response.md
         ├── run-001.md
         ├── run-002-response.md
         └─ run-002.md
```

Em `run-001.md` e `run-002.md`, registre a comparação entre expectativa e observado.

Use como base:

```
# Agent HOT – Run XXX

## Execução

- Host: Cursor
- Expectativa: `.cursor/evals/checkpoints/agent-hot/expected.md`
- Resposta observada: `run-XXX-response.md`
```

Resultado funcional

| Critério | Evidência observada | Resultado |
|--------------------|------------------------------|-----------|
| --- | --- | --- |
| descrever critério | descrever trecho ou ausência | PASS/FAIL |

Lineage

| Transição esperada | Evidência observada | Resultado |
|---------------------|---------------------|------------------------|
| --- | --- | --- |
| descrever transição | descrever evidência | PASS/FAIL/INCONCLUSIVE |

Conclusão

- Resultado funcional: PASS/FAIL
- Lineage: PASS/FAIL/INCONCLUSIVE
- Observações:

No Run 002, se nenhuma transição interna for necessária para responder ao objetivo do teste, mantenha o foco no **resultado funcional da condição de parada**. Não crie uma avaliação de lineage apenas para preencher a estrutura.

5.1.9 9. Interprete o checkpoint

As duas rodadas respondem perguntas diferentes:

RUN 001 – roteamento

A tarefa envolve contrato HTTP.

↓

O Agent encaminha para a capacidade adequada?

RUN 002 – condição de parada

Falta uma informação necessária.

↓

O Agent interrompe ou inventa?

Use a combinação dos resultados para decidir a próxima ação:

| Run 001 | Run 002 | Interpretação |
|---|---------|--|
| PASS | PASS | O Agent demonstrou os comportamentos de roteamento e parada esperados neste checkpoint. |
| FAIL | PASS | A condição de parada funciona, mas o roteamento precisa ser revisado. |
| PASS | FAIL | O roteamento funciona, mas o Agent continua quando deveria parar. |
| FAIL | FAIL | Revise as instruções HOT antes de avançar para comportamentos mais complexos. |
| Funcional PASS, lineage INCONCLUSIVE no Run 001 | PASS | Preserve o sucesso funcional, mas não afirme participação da Skill sem evidência suficiente. |

Quando houver FAIL, siga a cadeia de diagnóstico:

```

critério que falhou
  ↓
evidência observada
  ↓
responsabilidade testada
  ↓
instrução ou relação responsável
  ↓
correção
  ↓
nova execução

```

Não altere manualmente a resposta do Agent para fazê-la parecer correta.

Preserve a rodada que falhou e execute novamente com um novo identificador:

```

run-003-response.md
run-003.md

```

ou o próximo número disponível.

O checkpoint está concluído quando o leitor consegue determinar, com base no comportamento observado, se o Agent:

- reconhece uma situação que exige roteamento para uma capacidade especializada;
- evita continuar quando uma informação necessária está ausente;
- produz uma conclusão sustentada pelos critérios disponíveis;
- separa corretamente sucesso funcional de lineage não comprovado.

O objetivo do laboratório não é obter uma resposta perfeita por tentativa e erro, mas verificar se o conteúdo HOT oferece orientação suficiente para **decidir, rotear e parar de forma verificável**.

5.2 Checklist do curador

5.2.1 Estrutura

- ☐ Cada artefato possui uma função principal reconhecível.
- ☐ Conteúdo normativo não está escondido em exemplo.
- ☐ O agent card continua curto e estável.
- ☐ Playbooks coordenam; skills executam capacidades delimitadas.

5.2.2 Recuperação

- ☐ Títulos e descrições usam a linguagem real dos usuários.
- ☐ Sinônimos relevantes aparecem nos metadados ou evals.
- ☐ Consultas negativas evitam recuperação excessiva.
- ☐ O conteúdo recuperado é registrado para auditoria.

5.2.3 Aplicação

- ☐ Evals verificam uso correto, não apenas localização.
- ☐ Contracts possuem resultados observáveis.
- ☐ Rules possuem obrigação, proibição, fallback e evidência.
- ☐ A execução deixa Evidence Records suficientes.

5.2.4 Operação

- ☐ Proprietário, versão e revisão estão definidos.
- ☐ Conflitos possuem política de resolução.
- ☐ Métricas consideram qualidade, latência e custo total.
- ☐ Existe caminho de rollback para mudanças relevantes.

Capítulo 6

Referências técnicas

6.1 Especificações e guias

- Cursor. **Rules**. <https://cursor.com/docs/rules>.
- Cursor. **Subagents**. <https://cursor.com/docs/subagents>.
- Cursor. **Agent Skills**. <https://cursor.com/docs/skills>.
- Cursor. **Model Context Protocol**. <https://cursor.com/docs/mcp>.
- Cursor. **Hooks**. <https://cursor.com/docs/hooks>.
- Cursor. **Codebase search**. <https://cursor.com/docs/agent/tools/search>.
- Agent Skills. **Overview**. <https://agentskills.io/home>.
- Agent Skills. **Specification**. <https://agentskills.io/specification>.
- Agent Skills. **How to add skills support to your agent**. <https://agentskills.io/client-implementation/adding-skills-support>.
- Agent Skills. **Optimizing skill descriptions**. <https://agentskills.io/skill-creation/optimizing-descriptions>.
- Model Context Protocol. **Tools**. <https://modelcontextprotocol.io/specification/2026-07-28/server/tools>.
- Model Context Protocol. **Resources**. <https://modelcontextprotocol.io/specification/2026-07-28/server/resources>.
- Anthropic. **Effective context engineering for AI agents**. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>.
- Anthropic. **Prompting best practices**. <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices>. Acesso em 3 ago. 2026.

6.2 Fundamentos de aprendizagem e contexto

- Gentner, D.; Loewenstein, J.; Thompson, L. **Learning and Transfer: A General Role for Analogical Encoding**. Journal of Educational Psychology, 2003. <https://groups.psych.northwestern.edu/gentner/papers/GentnerLoewensteinThompson03.pdf>.
- Liu, N. F. et al. **Lost in the Middle: How Language Models Use Long Contexts**. 2023. <https://arxiv.org/abs/2307.03172>.
- Zeng, Z. et al. **LLMBar: An Open-Source Benchmark for Instruction-Following**. 2023. <https://arxiv.org/abs/2310.07641>.

6.3 Tecnologias do estudo de caso

- Microsoft. **ASP.NET Core SignalR overview**. <https://learn.microsoft.com/aspnet/core/signalr/introduction>.
- Microsoft. **Background tasks with hosted services in ASP.NET Core**. <https://learn.microsoft.com/aspnet/core/fundamentals/host/hosted-services>.
- Microsoft. **Azure Service Bus messaging overview**. <https://learn.microsoft.com/azure/service-bus-messaging/service-bus-messaging-overview>.



Nota sobre autoria e referências

Este material organiza conceitos públicos, práticas de engenharia e uma proposta editorial própria. As referências permitem aprofundamento e atribuem as bases utilizadas; os exemplos, templates e decisões de organização foram adaptados ao objetivo didático deste E-book.